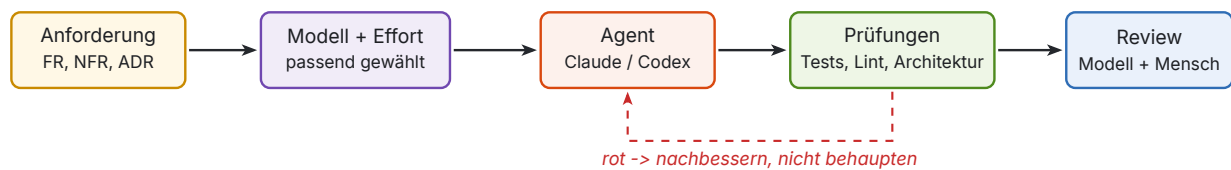


Claude und Codex im Software Engineering

Modelle und Reasoning-Effort richtig wählen, Halluzinationen vermeiden, Aufgabentreue sichern und genau das Gefragte in hoher Qualität liefern



Inhalt

Bevor es losgeht	4	7.2 Vier strukturelle Gegenmaßnahmen	20
Aufbau	4	8 Aufgabentreue: das Gefragte, nicht mehr	22
Stand und Haltbarkeit	4	8.1 Die Anweisung gegen Overengineering	22
Konventionen	4	8.2 Eine Aufgabenstellung, die Grenzen setzt	22
		8.3 Planen vor dem Handeln	23
		8.4 Fragen oder annehmen?	23
		8.5 Abweichungen sichtbar machen	23
TEIL I · GRUNDLAGEN		9 Goals und lang laufende Aufgaben	24
1 Wie Coding-Agenten arbeiten	6	9.1 Wie Goals funktionieren	24
1.1 Die Agentenschleife	6	9.2 Ist ein Goal ein guter Ansatz?	24
1.2 Warum Agenten zu viel oder das Falsche tun	6	9.3 Die Grenzen kennen	25
1.3 Die vier Hebel	7	9.4 Gute und schlechte Zielbedingungen	25
1.4 Kontextfenster und Verdichtung	7	9.5 Verwandte Mechanismen	25
2 Die Modelle im Überblick	8	10 ADRs und Anforderungen als Leitplanken	26
2.1 Anthropic: Claude	8	10.1 Was Spec-Driven Development will	26
2.2 OpenAI: Codex	9	10.2 Was bereits vorhanden ist	26
2.3 Die Staffelung nebeneinander	9	10.3 Anforderungen agententauglich schreiben	26
3 Reasoning-Effort verstehen	10	10.4 ADRs agententauglich schreiben	27
3.1 Was Effort bewirkt	10	10.5 Der Ablauf	28
3.2 Die Stufen bei Claude	10	10.6 Rückverfolgbarkeit ohne Zusatzaufwand	28
3.3 Die Stufen bei Codex	11	10.7 Architekturkonformität automatisch prüfen	29
3.4 Claude und Codex nebeneinander	11	10.8 Wann doch ein neues ADR entsteht	29
TEIL II · AUSWAHL IN DER PRAXIS		11 Qualitätssicherung im Ablauf	30
4 Einrichtung auf allen Oberflächen	12	11.1 Die Prüfpypamide	30
4.1 Claude: CLI, Desktop-App und VS Code	12	11.2 Taugen die Tests? Mutation Testing	30
4.2 Codex: CLI, ChatGPT-Desktop-App und IDE-Erweiterung	13	11.3 Prüfungen automatisch auslösen: Hooks	31
4.3 Kosten und Nutzungsvolumen im Abo	13	11.4 Berechtigungen und Sandbox	31
5 Welches Modell und welcher Effort wofür	15	11.5 Checkliste für das Review von KI-Änderungen	32
5.1 Die Eskalationsleiter	15	11.6 Wissen, wann Schluss ist	32
5.2 Entscheidungstabelle nach Aufgabentyp	15	12 Sicherheit beim Arbeiten mit Agenten	33
5.3 Wann Fable oder Astra ihr Geld wert sind	15	12.1 Das Bedrohungsmodell	33
5.4 Ein Modell für die Planung, eines für die Umsetzung	16	12.2 Die Schutzschichten	33
5.5 Am eigenen Code kalibrieren	16	12.3 Fremde Repositories	34
6 Claude und Codex als gegenseitige Prüfer	18	12.4 Geheimnisse und Zugangsdaten	34
6.1 Die Regeln	18	12.5 MCP-Server und Abhängigkeiten	34
6.2 Codex prüft eine Änderung von Claude	18	12.6 Unbeaufsichtigte Läufe	35
6.3 Claude prüft eine Änderung von Codex	18	13 Ein Durchlauf von Anfang bis Ende	36
6.4 Parallel arbeiten mit Worktrees	19	13.1 Schritt 1: Sitzung in einem eigenen Worktree	36
TEIL III · QUALITÄT		13.2 Schritt 2: Plan anfordern und korrigieren	36
7 Halluzinationen vermeiden	20	13.3 Schritt 3: Tests zuerst	37
7.1 Die typischen Formen in der Softwareentwicklung	20		

13.4 Schritt 4: Umsetzung als Goal	37
13.5 Schritt 5: Prüfen	37
13.6 Schritt 6: Taugen die Tests?	37
13.7 Schritt 7: Review durch Codex	38
13.8 Schritt 8: Menschliches Review und Com- mit	38
13.9 Was der Durchlauf zeigt	38

TEIL IV · UNTER DER HAUBE

14 System-Prompt und Inferenz-Einstellungen	39
14.1 Was bei jeder Anfrage an das Modell geht	39
14.2 Der System-Prompt	39
14.3 Inferenz-Einstellungen: wie aus Wahr- scheinlichkeiten Text wird	40
14.4 Was davon bei Claude und Codex einstell- bar ist	40
14.5 Kontextlänge	41
15 Lokale Modelle: Formate, Speicher und Sam- pling	42
15.1 Die Formate	42
15.2 Quantisierung und Speicherbedarf	43
15.3 Kontextlänge lokal einstellen	43
15.4 Sampling für lokale Modelle	44
15.5 Chat-Vorlage und System-Prompt lokal .	44
15.6 Lokale Modelle in Claude Code und Co- dex	45

ANHANG

A Vorlagen	46
A.1 Eine Datei für beide Werkzeuge	46
A.2 Aufgabenstellungen	47
A.3 Schnellreferenz	47
B Glossar	48

VERZEICHNISSE

Literaturverzeichnis	50
-----------------------------	-----------

Bevor es losgeht

Coding-Agenten wie Claude Code und Codex schreiben heute ganze Features, führen Migrationen durch und reparieren Tests. Ob das Ergebnis brauchbar ist, entscheidet sich aber weniger am Modell selbst als an vier Fragen: Welches Modell mit welchem Reasoning-Effort passt zur Aufgabe? Was weiß der Agent wirklich, und was vermutet er nur? Hält er sich an das, was gefragt war, oder baut er mehr? Und wie wird sichergestellt, dass am Ende nicht nur „fertig“ behauptet, sondern nachweislich richtig ist?

Dieses Handbuch beantwortet diese Fragen für die tägliche Softwareentwicklung. Der rote Faden lautet: **genau das Gefragte, in hoher Qualität, nicht mehr.** Overengineering ist dabei kein Schönheitsfehler, sondern ein Qualitätsproblem. Jede nicht angeforderte Abstraktion ist Code, der gelesen, getestet und gewartet werden muss.

Aufbau

- **Teil I, Grundlagen:** wie Coding-Agenten arbeiten, welche Modelle es Ende September 2026 bei Anthropic und OpenAI gibt und was Reasoning-Effort tatsächlich bewirkt.
- **Teil II, Auswahl in der Praxis:** Einrichtung auf allen Oberflächen (CLI, Desktop-Apps, VS Code), Entscheidungstabellen pro Aufgabentyp, eine Methode, Modell und Effort am eigenen Code zu kalibrieren, und das Zusammenspiel von Claude und Codex als gegenseitige Prüfer, auch parallel in Worktrees.
- **Teil III, Qualität:** Halluzinationen vermeiden, Aufgabentreue sichern, Goals sinnvoll einsetzen, mit ADRs und Anforderungen als Leitplanken arbeiten, ohne neue Artefakte einzuführen, die Qualitätssicherung im Ablauf einschließlich Mutation Testing, Sicherheit beim Arbeiten mit Agenten und zum Abschluss ein vollständiger Durchlauf von der Anforderung bis zum Commit.
- **Teil IV, Unter der Haube:** was bei jeder Anfrage an das Modell geht, System-Prompt, Inferenz-Einstellungen wie Temperature und Kontextlänge und warum sie bei den aktuellen Frontier-Modellen gesperrt sind, sowie lokale Modelle mit ihren Formaten (safetensors, MLX, GGUF), Speicherbedarf und Sampling.
- **Anhang:** Vorlagen für `CLAUDE.md`, `AGENTS.md` und Aufgabenstellungen sowie ein Glossar.

Stand und Haltbarkeit

Modellnamen, Standardwerte und Effort-Stufen ändern sich im Rhythmus weniger Monate. Alle konkreten Angaben entsprechen dem Stand Ende September 2026 und stammen aus den offiziellen Dokumentationen von Anthropic und OpenAI [1], [2], [3]. Das Literaturverzeichnis am Ende des Buchs nennt alle verwendeten Quellen im IEEE-Stil, die Kapitel verweisen mit Nummern in eckigen Klammern darauf. Die Prinzipien in Teil III sind davon unabhängig und bleiben gültig, wenn die Namen wechseln. Vor jeder Umstellung lohnt ein Blick in `/model` bzw. den Modellwähler der jeweiligen App.

Konventionen

Befehle, Ausgaben und Dateien erscheinen wie in den übrigen Handbüchern. In den Grafiken gilt:

 Claude	 Codex	 Modell / Effort	 Anforderung / ADR
 Prüfung / Test	 Review / Mensch	 Risiko / Fehler	

Das durchgängige Beispiel ist wieder die `notizen`-API aus den Handbüchern zu Docker und Secure API Design [4], [5]: FastAPI mit PostgreSQL, deren Anforderungen als Markdown-Dateien mit Akzeptanzkriterien unter `docs/anforderungen/` und deren Architekturentscheidungen als ADRs unter `docs/adr/` im Repository liegen.

MERKE

Ein Agent ist so gut wie die Prüfung, die sein Ergebnis bestehen muss. Das wichtigste Werkzeug gegen Halluzinationen und Overengineering ist deshalb keine Formulierung im Prompt, sondern eine **automatisch prüfbare Definition von „fertig“**: Tests aus Akzeptanzkriterien, Architekturregeln aus ADRs und ein Review gegen genau diese Vorgaben.

1 Wie Coding-Agenten arbeiten

Ein Coding-Agent ist ein Sprachmodell in einer Schleife: Er liest Dateien, plant, ändert Code, führt Befehle aus, liest deren Ergebnis und entscheidet, was als Nächstes zu tun ist [6], [7]. Wer diese Schleife versteht, versteht auch, wo Fehler entstehen und wo man eingreifen kann.

1.1 Die Agentenschleife

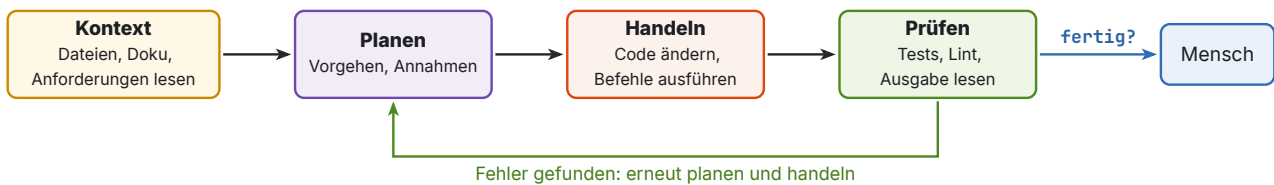


Abb. 1: Die Schleife eines Coding-Agenten. Qualität entsteht im Schritt „Prüfen“, nicht im Schritt „Handeln“.

Aus der Schleife folgen drei Einsichten, die sich durch das ganze Buch ziehen:

1. **Das Modell weiß nur, was im Kontext steht oder in seinen Trainingsdaten war.** Alles andere muss es über Werkzeuge beschaffen: Dateien öffnen, Befehle ausführen, Dokumentation lesen. Behauptungen ohne diesen Schritt sind Vermutungen (Kapitel 7).
2. **Die Prüfung ist der Qualitätshebel.** Ein Agent, der seine Änderung gegen Tests, Linter und Architekturregeln laufen lassen kann, korrigiert sich selbst. Ein Agent ohne Prüfmöglichkeit kann nur hoffen.
3. **Der Agent entscheidet selbst, wann er fertig ist,** außer man gibt ihm ein überprüfbares Kriterium vor (Kapitel 9 und 10).

1.2 Warum Agenten zu viel oder das Falsche tun

Sprachmodelle sind darauf trainiert, hilfreich zu sein. Bei Coding-Aufgaben äußert sich das in einem gut dokumentierten Muster: Anthropic beschreibt ausdrücklich, dass Opus-Modelle zum Overengineering neigen, also zusätzliche Dateien anlegen, unnötige Abstraktionen einführen oder nicht angeforderte Flexibilität einbauen [8]. Dazu kommen typische Formen des Abweichens:

Muster	Beispiel
Scope Creep	Beim Bugfix wird „gleich noch“ die umgebende Funktion umgebaut.
Spekulative Allgemeinheit	Ein Plugin-System für genau einen Anwendungsfall, eine Konfigurationsoption, die niemand braucht.
Defensives Übermaß	Fehlerbehandlung für Fälle, die nicht auftreten können, Validierung interner Aufrufe.
Stilfremde Lösungen	Neue Bibliothek, obwohl das Projekt schon eine für denselben Zweck nutzt.
Test-Gaming	Tests werden angepasst oder Sonderfälle hart codiert, bis sie grün sind.
Erfundene Anforderungen	Der Agent füllt Lücken in der Aufgabenstellung mit eigenen Annahmen, ohne sie zu nennen.

Diese Muster sind kein Zeichen eines schlechten Modells. Sie entstehen, wenn Aufgabe, Grenzen und Definition von „fertig“ nicht klar sind. Genau dort setzen die Kapitel 8 bis 10 an.

1.3 Die vier Hebel



Abb. 2: Modell und Effort bestimmen, was möglich ist. Kontext und Leitplanken bestimmen, was tatsächlich herauskommt.

Die Erfahrung aus der Praxis ist eindeutig: Ein mittleres Modell mit gutem Kontext und harten Leitplanken liefert bessere Ergebnisse als das stärkste Modell mit vager Aufgabenstellung und ohne Tests. Die Modellwahl ist wichtig, aber sie ist der Hebel, an dem man am wenigsten gewinnt, wenn die anderen drei fehlen.

1.4 Kontextfenster und Verdichtung

Alles, was der Agent gelesen und getan hat, liegt im Kontextfenster. Die aktuellen Modelle beider Hersteller arbeiten mit sehr großen Fenstern (bei Anthropic bis zu einer Million Token [1]). Irgendwann läuft das Fenster trotzdem voll, dann *verdichtet* das Werkzeug den bisherigen Verlauf zu einer Zusammenfassung (Claude Code: automatisch oder mit `/compact` [2], Codex: `/compact` [9]). Dabei gehen Details verloren.

Für die Praxis heißt das: **eine Aufgabe pro Sitzung**. Wer in einer Sitzung erst ein Feature baut, dann einen fremden Bug sucht und danach noch refaktoriert, bekommt einen Kontext voller irrelevanter Details. Mit `/clear` (Claude Code) bzw. einer neuen Unterhaltung beginnt die nächste Aufgabe sauber. Dauerhaft wichtige Informationen gehören nicht in den Chatverlauf, sondern in `CLAUDE.md` bzw. `AGENTS.md` und die Dokumente im Repository.

2 Die Modelle im Überblick

Beide Hersteller bieten eine Staffelung von Modellen: sehr fähig und teuer, ausgewogen, schnell und günstig. Dieses Kapitel ordnet die Ende September 2026 aktuellen Modelle ein [1], [3]. Die Namen ändern sich, die Staffelung bleibt.

2.1 Anthropic: Claude

Modell	Alias in Claude Code	Einordnung
Claude Fable 5.1	<code>fable</code>	Das fähigste allgemein verfügbare Modell, für Aufgaben größer als eine Sitzung. Hält lange autonome Sitzungen durch, untersucht vor dem Handeln und prüft seine Arbeit häufiger als kleinere Modelle [10].
Claude Opus 5.5	<code>opus</code>	Das Arbeitspferd für komplexes Engineering. Standardmodell in Claude Code auf Pro, Max, Team und Enterprise [2]. Laut Anthropic bei Standard-Effort <code>medium</code> mindestens so gut wie Opus 5 auf <code>high</code> , mit weniger Schritten und Token [11].
Claude Sonnet 5.5	<code>sonnet</code>	Für den täglichen Coding-Alltag, schneller und sparsamer als Opus. Arbeitet in der Ausführungsphase von <code>opusplan</code> [2].
Claude Haiku 4.5	<code>haiku</code>	Schnell und günstig für einfache Aufgaben. Wird von Claude Code intern für Hilfsaufgaben genutzt, etwa zur Bewertung von Goals (Kapitel 9) [12]. Unterstützt keine Effort-Stufen [1].

Über den Modellen steht die Mythos-Klasse (Claude Mythos 5.1). Fable 5.1 ist dasselbe zugrundeliegende Modell mit zusätzlichen Schutzmaßnahmen für Biologie, Cybersicherheit und KI-Forschung. Mythos selbst ist nicht öffentlich verfügbar.

Drei Besonderheiten sind für die Praxis wichtig:

- **Fable ist nirgends Standard** und muss ausdrücklich gewählt werden (`/model fable`). Je nach Plan läuft seine Nutzung über zusätzliches Nutzungsguthaben statt über das Planvolumen. Der Modellwähler zeigt dann „Requires usage credits“, und Claude Code fragt vor der ersten solchen Anfrage nach [2].
- **Alle aktuellen Modelle arbeiten mit einem Kontextfenster von einer Million Token**, auf allen Plänen, ohne Aufpreis für Token jenseits von 200.000 [1]. Claude Code verdichtet automatisch bei etwa 967.000 Token [2].
- **Sicherheitsklassifikatoren:** Fable, Opus 5.5 und Sonnet 5.5 prüfen Anfragen unter anderem auf Cybersicherheit. Schwachstellen im eigenen Quellcode zu finden ist erlaubt. Markiert der Klassifikator eine Anfrage, wiederholt Claude Code sie automatisch auf einem Ausweichmodell (von Fable und Opus 5.5 aus auf Opus 4.8) und zeigt einen Hinweis im Verlauf [2], [11].

AUS DER PRAXIS

Wer Security-Lehrmaterial, Pentest-Werkzeuge oder CTF-Aufgaben im Repository hat, sieht diese Umstellung häufig, oft schon bei der ersten Anfrage, weil `CLAUDE.md` und Verzeichnisnamen mitgeschickt werden. Das ist vorgesehenes Verhalten, keine Kontosperrung. Mit `claude --safe-mode` lässt sich prüfen, ob eigene Anpassungen die Ursache sind, und unter `/config` kann man einstellen, dass Claude Code vor dem Wechsel fragt („Switch models when a message is flagged“) [2].

2.2 OpenAI: Codex

Codex ist Ende September 2026 Teil der ChatGPT-Desktop-App (Bereiche *Work* und *Codex*), der Codex CLI, der IDE-Erweiterung, von ChatGPT im Web und von Codex Cloud. Bei Anmeldung mit einem ChatGPT-Abo empfiehlt OpenAI folgende Modelle [3]:

Modell	Kennung	Einordnung laut OpenAI
GPT-6 Astra	<code>gpt-6-astra</code>	Das fähigste Modell für komplexe Arbeit über Code, Apps und Recherche. Für die schwierigsten Aufgaben über viele Schritte und Werkzeuge.
GPT-6.1 Sol	<code>gpt-6.1-sol</code>	Nahe an Astra, aber günstiger. Empfohlen für komplexes Coding und agentische Abläufe, besonders für wiederholte, lang laufende Arbeit.
GPT-6 Luna	<code>gpt-6-luna</code>	Das effizienteste Modell für klare, wiederholbare Aufgaben mit hohem Volumen: Extraktion, Klassifikation, fokussiertes Coding.

GPT-6 Sol und die GPT-5.6-Familie (Sol, Terra, Luna) bleiben während der Einführung verfügbar. **GPT-5.5 wird am 14. Oktober 2026 aus ChatGPT und Codex entfernt**, GPT-5.4 und GPT-5.4 mini sind seit 31. August 2026 nicht mehr verfügbar. Wer diese Modelle in `config.toml`, eigenen Agenten oder Skripten eingetragen hat, muss umstellen. Verfügbarkeit hängt vom Plan, dem Client und dem Stand der Einführung ab [3], [13].

2.3 Die Staffelung nebeneinander

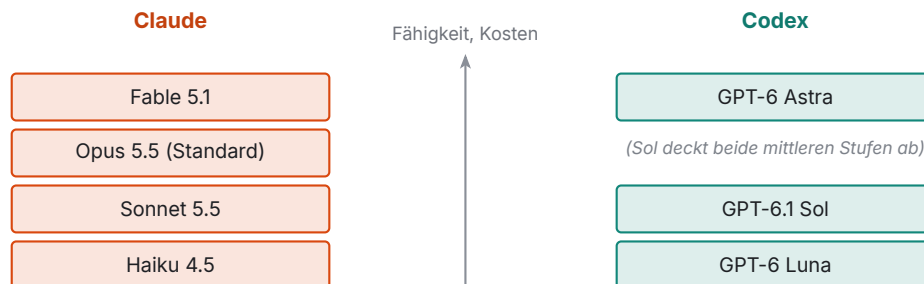


Abb. 3: Grobe Zuordnung der Stufen. Die Stufen sind keine gleichwertigen Paare, die Einordnung ersetzt keinen Vergleich an eigenen Aufgaben.

MERKE

Die Standardmodelle beider Werkzeuge (Opus 5.5 in Claude Code, GPT-6.1 Sol in Codex) sind für den größten Teil der Softwareentwicklung die richtige Wahl. Nach oben (Fable, Astra) wechselt man für lange, unklare oder besonders schwierige Aufgaben, nach unten (Sonnet, Haiku, Luna) für klar umrissene Arbeit mit hohem Volumen. Kapitel 5 macht das konkret.

3 Reasoning-Effort verstehen

Neben dem Modell gibt es einen zweiten, oft wirkungsvolleren Regler: den Reasoning-Effort. Er bestimmt, wie viel das Modell nachdenkt, bevor es antwortet oder handelt, und wie gründlich es seine Arbeit prüft.

3.1 Was Effort bewirkt

Die aktuellen Modelle beider Hersteller entscheiden bei jedem Schritt selbst, ob und wie lange sie nachdenken (*adaptive reasoning*) [3], [14]. Der Effort legt fest, wie großzügig sie das tun. Bei Claude lässt sich das Denken auf Opus 5.5, Sonnet 5.5 und Fable gar nicht mehr abschalten, der Effort ist der einzige Regler dafür [14], [15].

Anthropic beschreibt den Unterschied an konkreten Beobachtungen: Mit höherem Effort testete Claude mehr Randfälle und prüfte mehr seiner Arbeit, bevor es antwortete, traf aber auch mehr Entscheidungen selbst. Mit niedrigerem Effort lieferte es schneller einen Ausgangspunkt, was zu Arbeit passt, bei der man jedes Ergebnis prüft und den nächsten Schritt selbst steuert [2], [14].

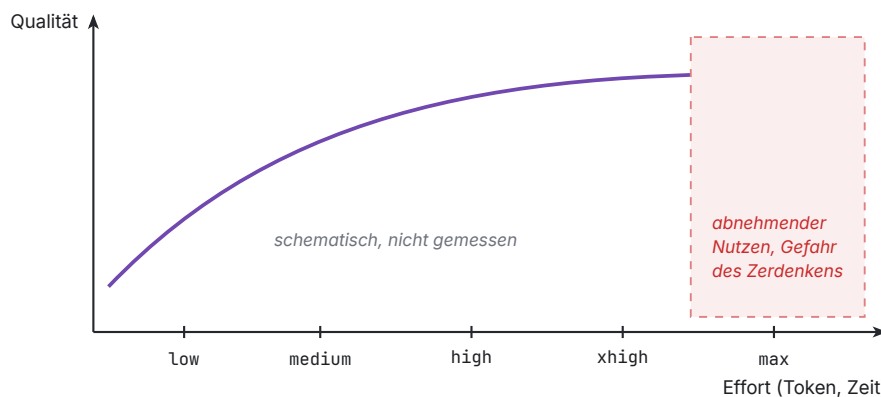


Abb. 4: Mehr Effort bringt anfangs viel, dann immer weniger. Auf der höchsten Stufe warnt Anthropic ausdrücklich vor Zerdenken [14].

3.2 Die Stufen bei Claude

Stufe	Wann (nach Claude-Code-Dokumentation [2])
low	Schneller Austausch, bei dem man jedes Ergebnis selbst prüft: Brainstorming, erster Entwurf, kleine Änderung wie eine Umbenennung.
medium	Standard auf Opus 5.5 und Sonnet 5.5. Tägliche Engineering-Arbeit mit klarem Umfang, etwa ein neues Feature.
high	Wo Verifikation zählt oder Randfälle wahrscheinlich sind, etwa ein Bugfix in bestehendem Code. Standard auf den übrigen Modellen.
xhigh	Tieferes Nachdenken bei höherem Verbrauch.
max	Schwere Probleme, die Claude ohne Rückfragen durcharbeiten soll, etwa das Suchen von Sicherheitslücken. Kann abnehmenden Nutzen bringen und zum Zerdenken neigen, vor breitem Einsatz testen.

Dazu kommen zwei Werkzeuge in Claude Code, die keine eigenen Stufen sind: **ultracode** ist eine Einstellung, mit der Claude für größere Aufgaben selbstständig Arbeitsabläufe mit mehreren Teilagenten plant. **ultrathink** im Prompt verlangt für genau diese eine Anfrage gründlicheres Nachdenken, ohne die Sitzungsstufe zu ändern. Formulierungen wie „denk gründlich nach“ haben dagegen keine Sonderbedeutung mehr [2].

ACHTUNG

Die Stufen sind pro Modell kalibriert. `medium` auf Opus 5.5 ist nicht dasselbe wie `medium` auf Opus 5. Anthropic empfiehlt beim Wechsel auf Opus 5.5 ausdrücklich, bei `medium` zu beginnen, statt die alte Stufe mitzunehmen. Wer weniger Nachdenken will, senkt zuerst den Effort: Das wirkt zuverlässiger als Anweisungen im Prompt [11].

3.3 Die Stufen bei Codex

In der ChatGPT-Desktop-App wählt man unter dem Eingabefeld eine *Power*-Voreinstellung, die Modell und Effort kombiniert (etwa „6 Sol Light“ oder „Astra Medium“), oder unter *Advanced* beides getrennt. In der CLI und in `config.toml` heißen die Stufen [3], [16]:

App	CLI / Konfiguration	Einsatz laut OpenAI
Light	<code>low</code>	schnelle, klar umrissene Aufgaben
Medium	<code>medium</code>	Balance, wenn mehr Planung nötig ist
High, Extra High	<code>high</code> , <code>xhigh</code>	schwierige Arbeit mit mehreren Schritten, Quellen oder Abwägungen
Max	<code>max</code>	mehr Denkzeit für eine einzelne, sehr schwere Aufgabe
Ultra	(eigene Option)	verteilt eine große Aufgabe auf parallele Teilagenten

OpenAI gibt Startwerte vor: für Luna **High**, für Astra **Light**, für GPT-6.1 Sol die voreingestellte Stufe des Clients [3]. Auch hier gilt: Stufen entsprechen sich zwischen Modellgenerationen nicht genau. Die meisten Aufgaben brauchen weder Max noch Ultra.

3.4 Claude und Codex nebeneinander

Absicht	Claude Code	Codex
schnell, ich prüfe jeden Schritt	<code>low</code>	Light / <code>low</code>
normale Feature-Arbeit	<code>medium</code> (Opus 5.5)	Standard von 6.1 Sol
Bugfix, Randfälle, Verifikation	<code>high</code>	High
schwierige Analyse, Architektur	<code>xhigh</code>	Extra High
allein durcharbeiten, sehr schwer	<code>max</code> (selten)	Max (selten)
große Aufgabe parallelisieren	<code>ultracode</code>	Ultra

Die Tabelle ordnet Absichten zu, keine gleich starken Einstellungen. Die sinnvollste Regel für beide Werkzeuge lautet: **mit dem Standard beginnen, erst den Effort erhöhen und erst danach das Modell wechseln.** Wechsel von Modell oder Effort mitten in der Sitzung können den Prompt-Cache ungültig machen und damit zusätzlich kosten, Claude Code warnt in diesem Fall [14], [17].

4 Einrichtung auf allen Oberflächen

Claude und Codex gibt es jeweils als Kommandozeilenwerkzeug, als Desktop-App und als Erweiterung für VS Code. Die gute Nachricht: Pro Hersteller teilen sich alle Oberflächen dieselbe Konfiguration. Wer sie einmal sauber einrichtet, hat überall dieselben Standards.



Abb. 5: Alle Oberflächen eines Herstellers lesen dieselbe Konfiguration. Projektdateien im Repository überschreiben persönliche Einstellungen.

4.1 Claude: CLI, Desktop-App und VS Code

Die Desktop-App führt im Bereich *Code* die Claude-Code-CLI selbst aus, die VS-Code-Erweiterung ebenso. Modell- und Effort-Einstellungen gelten deshalb überall gleich.

Befehl oder Einstellung	Wirkung [2]
<code>/model</code>	Modellwähler öffnen. <code>[Enter]</code> wechselt und speichert als Standard, <code>[S]</code> wechselt nur für diese Sitzung. Mit den Pfeiltasten lässt sich im Wähler auch der Effort verschieben.
<code>/model opus</code> , <code>/model fable</code>	direkt wechseln (Aliase <code>fable</code> , <code>opus</code> , <code>sonnet</code> , <code>haiku</code> , <code>opusplan</code>)
<code>/effort</code> , <code>/effort high</code> , <code>/effort auto</code>	Effort-Regler öffnen, Stufe setzen, auf den Modellstandard zurücksetzen. Gespeichert wird pro Modell .
<code>claude --model sonnet --effort low</code>	Modell und Effort nur für diese Sitzung
<code>ultrathink</code> im Prompt	gründlicheres Nachdenken nur für diese Anfrage
<code>/status</code> , Statuszeile	aktives Modell und Effort anzeigen; der Effort steht auch im Sitzungskopf („with low effort“)

Für Projekte, in denen alle Beteiligten mit denselben Vorgaben arbeiten sollen, gehört eine Projektdatei ins Repository:

`.claude/settings.json`

```

{
  "model": "opus",
  "effortLevel": "medium",
  "permissions": {
    "allow": ["Bash(uv run pytest:*)", "Bash(uv run ruff:*)", "Bash(uv run lint-imports)",
    "deny": ["Read(./env)", "Read(./secrets/**)"]
  }
}
  
```

`effortLevel` in einer Projektdatei gilt für jedes Modell, `max` wird dort nicht angenommen [2]. Persönliche Abweichungen (etwa `/effort high` für die eigene Sitzung) bleiben jederzeit möglich. Eine Kette von Ausweichmodellen für Überlastungsfälle lässt sich mit `"fallbackModel": ["claude-sonnet-5", "claude-haiku-4-5"]` festlegen [2].

opusplan : planen mit Opus, ausführen mit Sonnet

Der Alias `opusplan` nutzt im Plan-Modus Opus für Architektur- und Lösungsentscheidungen und wechselt für die Umsetzung automatisch auf Sonnet [2]. Das spart Nutzungsvolumen bei Aufgaben, deren Plan anspruchsvoll, deren Ausführung aber Routine ist. Bei Aufgaben mit kniffliger Umsetzung (nebenläufiger Code, komplexe Datenmigration) ist Opus durchgehend die bessere Wahl.

Teilagenten auf kleineren Modellen

Claude Code delegiert Teilaufgaben an *Subagents* [18]. Standardmäßig erben sie das Modell der Sitzung. Für Recherche, Suchen im Code oder das Ausführen von Tests genügt oft ein kleineres Modell. Das legt man in der Definition des Teilagenten fest, einschließlich des Effort:

```
.claude/agents/testlaeufer.md
```

```
---
name: testlaeufer
description: Führt Tests aus und fasst Fehlschläge knapp zusammen. Ändert keine Dateien.
tools: Bash, Read, Grep
model: haiku
---
Führe die angegebenen Tests aus. Berichte je Fehlschlag: Test, Fehlermeldung,
betroffene Datei und Zeile. Keine Vermutungen über Ursachen, keine Änderungen.
```

Global lässt sich ein Standard für alle Teilagenten über `CLAUDE_CODE_SUBAGENT_MODEL` setzen [18]. Achtung: Wer mit `/model` auf Opus wechselt, bevor Claude delegiert, schickt auch erbende Teilagenten auf Opus.

4.2 Codex: CLI, ChatGPT-Desktop-App und IDE-Erweiterung

Die ChatGPT-Desktop-App, die Codex CLI und die IDE-Erweiterung verwenden dieselbe Datei `config.toml` [16]. In der App wählt man Modell und Effort über den *Power*-Regler unter dem Eingabefeld oder unter *Advanced*, in der CLI mit `/model`. Max und Ultra erscheinen in der CLI unter *More reasoning...*; in der App muss Ultra gegebenenfalls unter *Settings* -> *Configuration* eingeschaltet werden.

```
~/codex/config.toml
```

```
model = "gpt-6.1-sol"
model_reasoning_effort = "medium"      # low | medium | high | xhigh | max
plan_mode_reasoning_effort = "high"    # im Plan-Modus gründlicher
```

```
codex                                # interaktiv mit den Standardwerten
codex -m gpt-6-astra                 # anderes Modell für diese Sitzung
codex -c model_reasoning_effort="high" # anderer Effort für diese Sitzung
codex exec -m gpt-6-luna "Formatiere die Changelog-Einträge einheitlich" # nicht interaktiv
```

Die Optionen der CLI (`-m`, `-c`, `exec`) beschreibt die Befehlsreferenz [9]. `plan_mode_reasoning_effort` [19] ist das Gegenstück zu `opusplan`: gründliches Planen, sparsameres Ausführen, allerdings auf demselben Modell. Wie bei Claude gehören gemeinsame Vorgaben in das Projekt (`.codex/config.toml`), persönliche in `~/codex/`.

ACHTUNG

Modellkennungen in `config.toml`, eigenen Agentendefinitionen und Skripten veralten. Nach dem 14. Oktober 2026 schlägt jede Konfiguration fehl, die noch `gpt-5.5` nennt [3]. Wer statt fester Versionen mit den Standardwerten des Clients arbeitet, ist davon nicht betroffen, verliert aber die Kontrolle darüber, wann ein neues Modell eingesetzt wird. Für Projekte mit reproduzierbarem Verhalten: Versionen festlegen und bewusst aktualisieren.

4.3 Kosten und Nutzungsvolumen im Abo

Mit einem Claude-Abo (Pro, Max) und einem ChatGPT-Abo wird nicht pro Token abgerechnet, sondern gegen ein Nutzungsvolumen pro Zeitfenster. Das verschiebt die Frage von „was kostet es“ zu „wie lange reicht es“:

- **Effort ist der größte Verbrauchstreiber.** Eine Sitzung auf `xhigh` verbraucht ein Vielfaches von `medium`. Hohe Stufen deshalb gezielt für einzelne Aufgaben einsetzen, nicht als Dauereinstellung.
- **Große Modelle verbrauchen mehr pro Token.** Fable und Astra für Aufgaben, die sie brauchen, Teilagenten auf Haiku oder Luna für Suchen, Zusammenfassen und Testläufe.
- **Kontext kostet bei jeder Anfrage.** Ein aufgeblähter Verlauf wird bei jedem Schritt erneut verarbeitet. Eine Aufgabe pro Sitzung spart mehr als jede Modellwahl.

5 Welches Modell und welcher Effort wofür

Die Modellwahl folgt einer einfachen Logik: **So viel wie nötig, so wenig wie möglich**. Mehr Rechenleistung macht eine unklare Aufgabe nicht klarer und eine fehlende Prüfung nicht überflüssig. Sie hilft dort, wo das Problem selbst schwierig ist.

5.1 Die Eskalationsleiter



nur eine Stufe höher, wenn die vorige nachweislich nicht reicht

Abb. 6: Bevor mehr Rechenleistung hilft, hilft fast immer eine bessere Aufgabenstellung.

Die Reihenfolge ist Absicht. Schritt 2 steht vor Schritt 3, weil die häufigste Ursache schlechter Ergebnisse eine unklare Aufgabe ist, nicht ein zu schwaches Modell. Schritt 5 steht am Ende, weil auch das stärkste Modell an einer Aufgabe scheitert, die zu groß oder zu vage ist.

5.2 Entscheidungstabelle nach Aufgabentyp

Aufgabe	Claude Code	Codex
Frage zum Code, Erklärung	Standard, low bis medium	Sol, Light
Umbenennen, kleine lokale Änderung	Sonnet oder Opus, low	Sol oder Luna, Light
Feature mit klaren Akzeptanzkriterien	Opus 5.5, medium	6.1 Sol, Standard
Bugfix in bestehendem Code	Opus 5.5, high	6.1 Sol, High
Tests zu vorhandenen Kriterien schreiben	Opus oder Sonnet, medium	Sol, Medium
Architekturentwurf, ADR-Entwurf	Opus 5.5 xhigh oder Fable	Astra, Medium bis High
Unklare Fehlerursache, Ausfallanalyse	Fable, high	Astra, High
Große Migration, viele Dateien	Fable oder Opus + Goal (Kap. 9)	6.1 Sol + Goal
Code-Review eines Diffs	Opus 5.5, high	6.1 Sol, High
Sicherheitsreview	Opus 5.5 oder Fable, high bis max	Astra, High bis Max
Mechanische Massenänderung	Teilagent auf Haiku / Sonnet	Luna, High
Doku, Changelog, Commit-Texte	Sonnet, low	Luna

Die Tabelle ist ein Startpunkt, kein Gesetz. Zwei Beobachtungen aus der Dokumentation beider Hersteller rechtfertigen die Standardwahl für den Großteil der Arbeit: Opus 5.5 auf medium erreicht laut Anthropic bei Coding-Aufgaben mindestens das Niveau von Opus 5 auf high, auf mehreren Coding-Evaluationen kommt sogar low nahe heran [2], [11]. OpenAI empfiehlt GPT-6.1 Sol für komplexes Coding und bezeichnet Astra als Wahl für die anspruchsvollsten Aufgaben [3].

5.3 Wann Fable oder Astra ihr Geld wert sind

Anthropic beschreibt die Stärken von Fable sehr konkret [10]: Es untersucht vor dem Handeln, prüft seine Arbeit selbst und hält lange Sitzungen durch. Die Empfehlungen für den Einsatz lauten:

- **Das Ergebnis beschreiben, nicht die Schritte.** Fable plant den Weg selbst.
- **Unklare Probleme übergeben:** Ursachenanalyse, Fehlersuche bei Ausfällen, Architekturentscheidungen.
- **Auf Prüf-Erinnerungen verzichten.** Fable prüft ohnehin, Hinweise wie „teste das“ sind meist überflüssig.
- **Größere Pakete übergeben,** die man sonst zerlegen würde.

Für Astra formuliert OpenAI ähnlich: Es ist die Wahl für vollständige Abläufe, die anhaltendes Nachdenken und Urteilsvermögen brauchen, und man soll ihm die Quellen, Vorlagen, Einschränkungen und Prüfungen mitgeben, die ein brauchbares Ergebnis definieren [3].

Umgekehrt lohnt sich die Spitzenklasse nicht für Aufgaben, deren Schwierigkeit in der Menge liegt statt in der Tiefe, und nicht für schlecht spezifizierte Aufgaben. Ein Spitzenmodell mit vager Aufgabe liefert eine besonders gut begründete Lösung für das falsche Problem.

5.4 Ein Modell für die Planung, eines für die Umsetzung

Muster	Umsetzung
Planen stark, ausführen sparsam	Claude: <code>opusplan</code> oder Plan-Modus auf Opus, dann <code>/model sonnet</code> . Codex: <code>plan_mode_reasoning_effort = "high"</code> bei normalem Effort für die Ausführung.
Hauptagent stark, Hilfsarbeit klein	Teilagenten mit <code>model: haiku</code> (Claude) bzw. Luna für Suche, Tests, Zusammenfassungen.
Zweitmeinung während der Arbeit	Claude Code kann über das <i>Advisor</i> -Werkzeug mitten in der Aufgabe ein zweites Modell konsultieren, statt starr an der Plangrenze zu wechseln [2].
Umsetzen mit einem, prüfen mit dem anderen	Claude implementiert, Codex prüft, oder umgekehrt (Kapitel 6).

TIPP

Eine Faustregel für die Wahl des Effort: Wenn man bereit ist, jedes Zwischenergebnis selbst anzusehen und zu steuern, reicht ein niedriger Effort. Wenn der Agent länger allein arbeiten soll, lohnt ein höherer, weil er dann selbst gründlicher prüfen muss.

5.5 Am eigenen Code kalibrieren

Beide Hersteller betonen dasselbe: Effort-Stufen und Modelle sollte man an den eigenen Aufgaben testen, statt Empfehlungen zu übernehmen. Anthropic rät bei Opus 5.5 ausdrücklich, mehrere Stufen gegen eigene Auswertungen laufen zu lassen, OpenAI empfiehlt, eine vertraute Aufgabe auf einer niedrigeren Stufe zu probieren und dann anzupassen [3], [11]. Mit wenig Aufwand lässt sich das systematisch machen, und zwar ohne neues Dokument: Das Testmaterial steckt bereits in der Git-Historie [20].

Die Methode

1. **Aufgaben aus der Historie wählen.** Fünf bis acht abgeschlossene Änderungen, die typisch für das Projekt sind: zwei Features, zwei Bugfixes, ein Refactoring, eine Migration. Jede muss eine Anforderung mit Akzeptanzkriterien und im Commit mitgelieferte Tests haben.
2. **Die echten Tests als Schiedsrichter nutzen.** Der Agent bekommt den Stand **vor** dem Commit und die Anforderung, aber nicht die Tests. Nach seinem Lauf werden die Tests aus dem Original-Commit eingespielt und ausgeführt. Sie prüfen unabhängig davon, was der Agent selbst getestet hat.
3. **Wenige Konfigurationen vergleichen,** etwa Opus 5.5 auf `low`, `medium` und `high` sowie GPT-6.1 Sol auf Light, Medium und High.
4. **Jede Kombination zwei- bis dreimal laufen lassen.** Die Ergebnisse streuen, ein einzelner Lauf sagt wenig.
5. **Vier Werte festhalten:** Anteil bestandener Originaltests, Diff-Größe (Hinweis auf Overengineering), Laufzeit und Verbrauch.

Ein Skript statt eines Dokuments

scripts/kalibrieren.sh (Skizze)

```
#!/usr/bin/env bash
# Aufruf: scripts/kalibrieren.sh <commit> <FR-Datei> <name> <agent-befehl...>
set -euo pipefail
ZIEL="$1"; FR="$2"; NAME="$3"; shift 3
BASIS="$(git rev-parse "$ZIEL^")"
WT="..../kal-$NAME"

git worktree add --detach "$WT" "$BASIS" >/dev/null
cp "$FR" "$WT/AUFGABE.md"
cd "$WT"
START=$(date +%s)
"$@" "Setze AUFGABE.md um. Halte dich an AGENTS.md. Schreibe eigene Tests." \
  > agent.log 2>&1 || true
DAUER=$(( $(date +%s) - START ))

git diff --stat | tail -1                # Diff-Größe
git checkout "$ZIEL" -- tests/           # Originaltests einspielen
uv run pytest -q 2>&1 | tail -1          # bestanden / fehlgeschlagen
echo "$NAME: ${DAUER}s"
cd - >/dev/null && git worktree remove --force "$WT"
```

```
scripts/kalibrieren.sh a1b2c3d docs/anforderungen/FR-009.md opus-medium \
  claude -p --model opus --effort medium --permission-mode acceptEdits
scripts/kalibrieren.sh a1b2c3d docs/anforderungen/FR-009.md sol-medium \
  codex exec -m gpt-6.1-sol -c model_reasoning_effort="medium" -s workspace-write
```

Auswerten

Konfiguration	Originaltests	Diff	Dauer	Beobachtung
Opus 5.5 low	7/8	+84	3 min	ein Randfall fehlt
Opus 5.5 medium	8/8	+91	5 min	passend
Opus 5.5 high	8/8	+143	9 min	zusätzliche Hilfsklasse
6.1 Sol Medium	8/8	+88	6 min	passend

Die Tabelle zeigt ein erfundenes Beispiel, aber ein typisches Muster: Die niedrigste Stufe verfehlt vereinzelt Randfälle, die mittlere trifft, die hohe liefert dasselbe Ergebnis mit mehr Code und mehr Zeit. Gewählt wird **die günstigste Konfiguration, die alle Originaltests zuverlässig besteht, ohne den Diff aufzublähen**. Den Verbrauch liest man bei Claude Code aus der Ausgabe von `-p --output-format json`, bei Codex aus `/usage` bzw. der Nutzungsübersicht des Kontos.

TIPP

Eine Kalibrierung lohnt sich einmal pro Projekt und dann wieder, wenn ein neues Modell erscheint. Bei Modellwechseln, etwa dem Ende von GPT-5.5 in Codex am 14. Oktober 2026 oder dem Wechsel von Opus 5 auf Opus 5.5, zeigt dieselbe Handvoll Aufgaben innerhalb einer Stunde, ob die bisherigen Stufen noch passen [3], [15].

6 Claude und Codex als gegenseitige Prüfer

Ein Modell, das seinen eigenen Code prüft, übersieht gerne dieselben Dinge, die es beim Schreiben übersehen hat. Ein zweites Modell eines anderen Herstellers bringt eine unabhängige Perspektive, ähnlich wie ein Review durch eine Kollegin. Mit beiden Abos lässt sich das ohne Mehrkosten in den Ablauf einbauen.

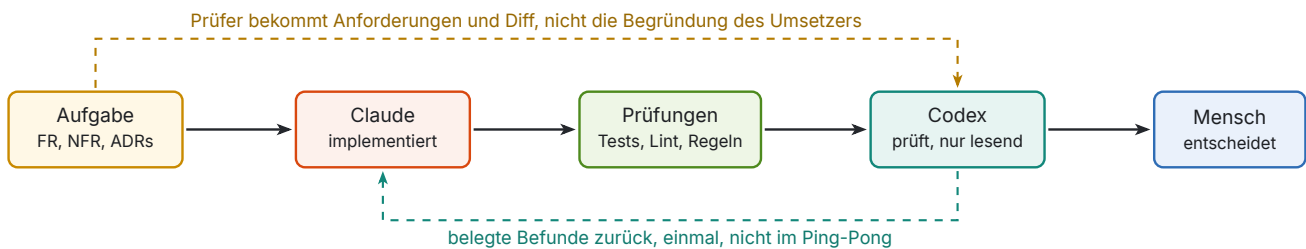


Abb. 7: Zwei Modelle, klare Rollen: Einer schreibt, einer prüft, der Mensch entscheidet.

6.1 Die Regeln

1. **Der Prüfer arbeitet nur lesend.** In Codex mit Sandbox `read-only` [21], in Claude Code als Teilagent ohne Schreibwerkzeuge [18]. Ein Prüfer, der „gleich selbst korrigiert“, ist ein zweiter Umsetzer, und dann weiß niemand mehr, wessen Änderung was bewirkt hat.
2. **Der Prüfer bekommt die Vorgaben, nicht die Begründung.** Anforderungen, Akzeptanzkriterien, relevante ADRs und den Diff. Die Erklärung des Umsetzers, warum alles richtig ist, verzerrt das Urteil.
3. **Befunde brauchen Belege:** Datei, Zeile, betroffene Anforderung oder ADR, konkreter Fall, in dem es schiefgeht. Befunde ohne Beleg werden verworfen.
4. **Ein Durchgang, kein Ping-Pong.** Der Umsetzer arbeitet die belegten Befunde einmal ab, danach entscheidet ein Mensch. Wechselseitiges Korrigieren über viele Runden produziert Überarbeitung um ihrer selbst willen.
5. **Prüfen, was geprüft werden soll:** Aufgabentreue (wurde mehr als verlangt geändert?), Abdeckung der Akzeptanzkriterien, ADR-Konformität, Fehler. Stil und Geschmack regeln Formatierer und Linter, nicht Modelle.

6.2 Codex prüft eine Änderung von Claude

```

codex exec -m gpt-6.1-sol -c model_reasoning_effort="high" -s read-only \
  "Prüfe den Diff gegenüber main (git diff main...HEAD) gegen docs/anforderungen/FR-012.md
  und docs/adr/0003-schichtenarchitektur.md. Melde nur belegte Befunde mit Datei, Zeile
  und betroffener Anforderung oder ADR: (1) Akzeptanzkriterien ohne Test, (2) Änderungen,
  die über FR-012 hinausgehen, (3) Verstöße gegen den ADR, (4) Fehler. Keine Stilfragen,
  keine Verbesserungsvorschläge ohne konkreten Fehler. Ändere keine Dateien."
  
```

In der interaktiven Codex-Sitzung und der Desktop-App erledigt `/review` eine Prüfung des aktuellen Arbeitsstands [9]. Für die Prüfung gegen Anforderungen und ADRs ist die ausdrückliche Aufgabenstellung oben trotzdem besser, weil sie festlegt, **wogegen** geprüft wird.

6.3 Claude prüft eine Änderung von Codex

```

.claude/agents/pruefer.md
  
```

```

---
name: pruefer
  
```

```

description: Prüft Diffs gegen Anforderungen und ADRs. Ändert keine Dateien.
tools: Read, Grep, Glob, Bash
model: opus
effort: high
---
Prüfe die angegebene Änderung ausschließlich gegen die genannten Anforderungen
(docs/anforderungen/) und ADRs (docs/adr/). Melde nur Befunde mit Beleg:
Datei, Zeile, betroffene ID, konkreter Fehlerfall. Kategorien: fehlende Tests
für Akzeptanzkriterien, Änderungen außerhalb des Auftrags, ADR-Verstöße, Fehler.
Keine Stilfragen, keine Vorschläge ohne konkreten Fehler, keine Änderungen.

```

ACHTUNG

Zwei Modelle können sich auch gemeinsam irren, etwa wenn beide dieselbe veraltete Bibliotheksversion annehmen. Das zweite Modell ersetzt deshalb weder die automatischen Prüfungen noch das menschliche Review. Es fängt eine zusätzliche Fehlerklasse ab: die blinden Flecken des Umsetzers.

6.4 Parallel arbeiten mit Worktrees

Zwei Agenten im selben Arbeitsverzeichnis überschreiben sich gegenseitig Dateien und Testergebnisse. Die Lösung sind Git-Worktrees: zusätzliche Arbeitsverzeichnisse desselben Repositories mit je eigenem Branch (Git-Handbuch, Kapitel 10 [20]; Referenz [22]). Beide Werkzeuge unterstützen sie direkt:

- **Claude Code:** `claude --worktree fr-012` (kurz `-w`) legt ein Arbeitsverzeichnis unter `.claude/worktrees/fr-012/` mit eigenem Branch an und startet die Sitzung darin. Dateien, die Git ignoriert, aber zum Arbeiten nötig sind (etwa `.env` für die Entwicklung), trägt man in `.worktreeinclude` ein [23]. Die Desktop-App bietet dasselbe über die Oberfläche.
- **Codex:** Die Desktop-App hat einen Worktree-Modus, der für einen Thread ein eigenes Arbeitsverzeichnis anlegt. In der CLI ist seit Version 0.154.0 ein experimenteller Schalter `--worktree` für abgezweigte Sitzungen verfügbar [13].
- **Manuell** funktioniert es mit beiden Werkzeugen: `git worktree add ../notizen-fr-014 -b feature/fr-014`, dann den Agenten im neuen Verzeichnis starten.

Typische Aufteilungen: Claude setzt FR-012 um, Codex parallel FR-014, jeweils in einem eigenen Worktree. Oder Codex prüft den fertigen Branch von Claude in einem eigenen Worktree, während Claude bereits am nächsten Auftrag arbeitet. Wichtig bleibt die Git-Regel: Ein Branch kann nur in einem Worktree gleichzeitig ausgecheckt sein [22].

7 Halluzinationen vermeiden

Eine Halluzination ist eine Behauptung, die plausibel klingt, aber nicht belegt ist [24]. In der Softwareentwicklung sind die gefährlichsten Halluzinationen keine erfundenen Fakten über die Welt, sondern stille Annahmen über den eigenen Code: eine Funktion, die es nicht gibt, ein Parameter, den die installierte Bibliotheksversion nicht kennt, oder die Aussage „alle Tests laufen“, ohne dass sie gelaufen sind.

Die aktuellen Modelle halluzinieren deutlich seltener als frühere. Anthropic beschreibt Opus 5.5 als viel seltener falsch bei Zahlen und Quellen [11], OpenAI hebt bei GPT-6 Sol die höhere Faktenzuverlässigkeit hervor. Das Risiko sinkt, verschwindet aber nicht. Die Gegenmaßnahmen sind deshalb vor allem strukturell.

7.1 Die typischen Formen in der Softwareentwicklung

Form	Beispiel	Gegenmaßnahme
Erfundene API	<code>client.fetch_all(retry=3)</code> existiert nicht	Code gegen die installierte Version prüfen lassen, Typprüfung und Tests
Falsche Version	Syntax einer älteren oder neueren Bibliotheksversion	Lockfile als Quelle nennen, aktuelle Doku bereitstellen
Aussagen über ungeöffneten Code	„Die Funktion validiert bereits die Eingabe“	Dateien vor jeder Aussage öffnen lassen
Erfundenes Ergebnis	„Tests laufen durch“, ohne Ausführung	nur Befehlsausgaben als Beleg akzeptieren, CI als letzte Instanz
Erfundene Anforderung	Annahmen über gewünschtes Verhalten werden stillschweigend umgesetzt	Annahmen offenlegen lassen, bei Unklarheit fragen
Test-Gaming	Sonderfall hart codiert, damit ein Test grün wird	ausdrückliches Verbot, Review des Diffs, Tests schützen
Veraltetes Wissen	Werkzeuge und Modelle nach dem Trainingsstand	Websuche oder Doku-Zugang, Stand datieren

7.2 Vier strukturelle Gegenmaßnahmen

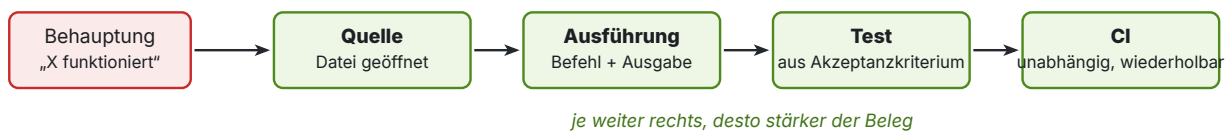


Abb. 8: Eine Behauptung des Agenten ist erst so viel wert wie der stärkste Beleg dahinter.

1. Erst lesen, dann sprechen

Anthropic empfiehlt für agentisches Coding ausdrücklich eine Anweisung, die Spekulation über ungeöffneten Code verbietet: Dateien, auf die sich eine Frage bezieht, müssen vor der Antwort gelesen werden, und Aussagen über die Codebasis sind nur nach Untersuchung erlaubt [8]. Sinngemäß gehört das in jede

CLAUDE.md und AGENTS.md :

Auszug aus CLAUDE.md / AGENTS.md

Belege statt Vermutungen

- Triff keine Aussage über Code, den du nicht geöffnet hast. Lies betroffene

Dateien, bevor du antwortest oder änderst.

- Bibliotheks-APIs gegen die installierte Version prüfen (uv.lock), nicht aus dem Gedächtnis. Bei Unsicherheit die Signatur nachsehen oder nachfragen.
- "Läuft", "grün", "behoben" nur mit der Ausgabe des ausgeführten Befehls.
- Wenn du etwas nicht weißt oder nicht prüfen kannst, sag es.

2. Werkzeuge statt Gedächtnis

Aktuelle Modelle sind darauf trainiert, Werkzeuge zu nutzen, statt zu raten. Rückmeldungen aus der Umgebung, etwa Testergebnisse oder Befehlsausgaben, sind dabei der Anker, an dem der Agent seinen Fortschritt prüft [6], [7]. Das funktioniert nur, wenn die Werkzeuge verfügbar und erlaubt sind: Tests, Typprüfung und Linter müssen ohne Rückfrage ausführbar sein (siehe die Berechtigungen in Kapitel 4). Für aktuelle Dokumentation von Bibliotheken helfen Websuche oder ein Dokumentations-Server über MCP. OpenAI empfiehlt für OpenAI-bezogene Fragen etwa, Codex per `AGENTS.md` auf den Dokumentations-Server der eigenen Entwicklerdoku zu verweisen [25].

3. Tests als Orakel, nicht als Hürde

Ein Test, der aus einem Akzeptanzkriterium abgeleitet wurde, ist die zuverlässigste Prüfung, ob eine Behauptung stimmt. Er ist aber nur dann ein Orakel, wenn er nicht nachträglich an den Code angepasst wird. Anthropic führt Test-Gaming ausdrücklich als Verhalten auf, gegen das man anweisen sollte: Lösungen sollen für alle gültigen Eingaben funktionieren, nicht nur für die Testfälle, und fehlerhafte Tests sollen gemeldet statt umgangen werden [8].

Auszug aus `CLAUDE.md` / `AGENTS.md`

Tests

- Schreibe eine allgemeine Lösung, die für alle gültigen Eingaben stimmt, nicht nur für die Testfälle. Keine hart codierten Sonderfälle für Tests.
- Ändere bestehende Tests nur, wenn die Aufgabe das ausdrücklich verlangt. Hältst du einen Test für falsch, melde es mit Begründung, statt ihn anzupassen.

4. Die CI hat das letzte Wort

Selbstauskünfte eines Agenten sind keine Nachweise. Maßgeblich ist, was die Pipeline aus dem Git-Handbuch (Kapitel 13 bis 15 [20]) unabhängig und wiederholbar feststellt. Ein Agent, der „fertig“ meldet, während die CI rot ist, hat nicht gelogen, er hat sich geirrt. Genau deshalb braucht es eine Instanz, die er nicht beeinflusst.

TIPP

Zwei kurze Fragen nach jeder größeren Änderung decken viele Halluzinationen auf: „Welche deiner Aussagen hast du mit welchem Befehl geprüft?“ und „Welche Annahmen hast du getroffen, die nicht in der Aufgabe standen?“ Ein gutes Modell beantwortet beide ehrlich, und die Antworten zeigen, wo nachgeprüft werden muss.

8 Aufgabentreue: das Gefragte, nicht mehr

Das Ziel ist genau die angeforderte Änderung, vollständig und sauber umgesetzt, ohne zusätzliche Abstraktionen, Optionen oder „Verbesserungen“. Das ist schwerer, als es klingt, weil Modelle dazu neigen, hilfsbereit mehr zu tun. Die Lösung liegt in drei Dingen: einer klaren Aufgabe, festen Grenzen und einer Prüfung, die Abweichungen sichtbar macht.

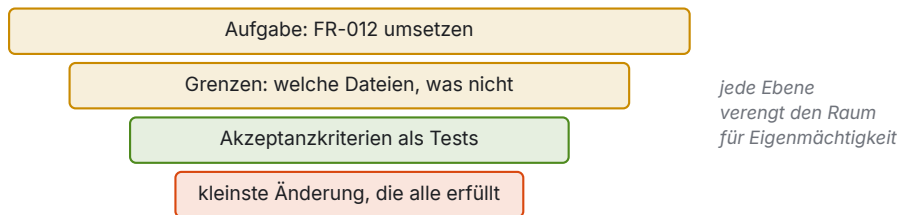


Abb. 9: Vom Auftrag zur kleinsten ausreichenden Änderung.

8.1 Die Anweisung gegen Overengineering

Anthropic veröffentlicht in den Prompting-Richtlinien einen Textbaustein, der Overengineering ausdrücklich adressiert [8]. Übersetzt und für ein Python-Projekt angepasst, gehört er in `CLAUDE.md` und `AGENTS.md`:

Auszug aus `CLAUDE.md` / `AGENTS.md`

Umfang

Vermeide Overengineering. Ändere nur, was direkt verlangt oder zwingend nötig ist.

- Umfang: Keine zusätzlichen Features, kein Refactoring und keine "Verbesserungen" über den Auftrag hinaus. Ein Bugfix braucht keine Aufräumarbeiten daneben.
- Doku: Keine Docstrings, Kommentare oder Typannotationen in Code, den du nicht geändert hast. Kommentare nur, wo die Logik nicht selbsterklärend ist.
- Fehlerbehandlung: Validiere an Systemgrenzen (Benutzereingaben, externe APIs), nicht bei internen Aufrufen. Keine Fallbacks für Fälle, die nicht eintreten können.
- Abstraktionen: Keine neuen Klassen, Schichten oder Konfigurationsoptionen für einen einzigen Anwendungsfall. Drei ähnliche Zeilen sind besser als eine voreilige Abstraktion.
- Abhängigkeiten: Keine neuen Pakete ohne ausdrückliche Zustimmung. Nutze, was im Projekt bereits vorhanden ist.

Wenn du eine Verbesserung außerhalb des Auftrags für wichtig hältst, nenne sie am Ende in einem Satz, statt sie umzusetzen.

Der letzte Satz ist wichtig. Er nimmt dem Modell den Druck, eine gute Idee sofort umzusetzen, und macht sie trotzdem sichtbar. So bleibt die Entscheidung beim Menschen.

8.2 Eine Aufgabenstellung, die Grenzen setzt

Unscharf

Füge Tags zu den Notizen hinzu.

Offen bleibt: Datenmodell, API-Form, Suche nach Tags, Umbenennen, Grenzen, Migration. Das Modell füllt alle Lücken selbst, meist großzügig.

Scharf

Setze FR-012 (`docs/anforderungen/FR-012.md`) um.

Beachte ADR-0003 und ADR-0007.

Nur `app/notizen/`, `app/db/` und `tests/`.

Keine neuen Abhängigkeiten.

Fertig: alle Akzeptanzkriterien als Tests grün, ruff und lint-imports sauber.

Nicht Teil der Aufgabe: Suche nach Tags.

Die scharfe Fassung ist nicht länger, weil sie mehr erklärt, sondern weil sie auf vorhandene Dokumente verweist und festlegt, **was nicht** dazugehört. Der Abschnitt „Nicht Teil der Aufgabe“ ist einer der wirksamsten Sätze gegen Scope Creep.

8.3 Planen vor dem Handeln

Beide Werkzeuge haben einen Plan-Modus, in dem der Agent liest und plant, aber nichts ändert (Claude Code: `⬆` `Tab` bis „plan mode“ bzw. Berechtigungsmodus `plan` [26]; Codex: `/plan` [9]). Für alles, was über eine lokale Änderung hinausgeht, lohnt sich der Ablauf:

1. Plan anfordern, der die betroffenen Anforderungen und ADRs mit ID nennt, die zu ändernden Dateien aufzählt und die getroffenen Annahmen offenlegt.
2. Plan prüfen. Hier ist Overengineering am billigsten zu korrigieren: Eine überflüssige Abstraktion im Plan kostet einen Satz, im Code eine Stunde.
3. Umsetzung freigeben.

Der Plan ist ein Gesprächsergebnis, kein neues Dokument. Er muss nirgends abgelegt werden.

8.4 Fragen oder annehmen?

Ein Agent, der bei jeder Kleinigkeit fragt, ist lästig. Einer, der nie fragt, trifft stille Annahmen. Eine brauchbare Regel für `CLAUDE.md` und `AGENTS.md`:

Unklarheiten

Wenn eine Unklarheit das Ergebnis wesentlich ändert (Datenmodell, API, Sicherheit, Verhalten für Benutzer), frage vor der Umsetzung.
Kleine Unklarheiten löst du mit der naheliegendsten Annahme und nennst sie am Ende in der Zusammenfassung.

8.5 Abweichungen sichtbar machen

Aufgabentreue lässt sich prüfen, nicht nur anweisen:

- **Diff-Größe beobachten.** Ein Bugfix mit 400 geänderten Zeilen in zwölf Dateien ist verdächtig. `git diff --stat` nach jeder Aufgabe zeigt das sofort.
- **Geänderte Pfade gegen die Grenzen prüfen.** Änderungen außerhalb der vereinbarten Verzeichnisse fallen im Review oder per Skript auf.
- **Neue Abhängigkeiten prüfen.** Eine Änderung an `pyproject.toml` oder `uv.lock` ohne Auftrag ist ein Befund.
- **Der Prüfer aus Kapitel 6 fragt ausdrücklich nach Änderungen außerhalb des Auftrags.**

MERKE

Aufgabentreue ist keine Frage des Modells, sondern des Vertrags. Ein klarer Auftrag mit Verweis auf Anforderung und ADRs, ausdrücklichen Grenzen, einer prüfbaren Definition von „fertig“ und einem Abschnitt „Nicht Teil der Aufgabe“ erzielt mit jedem aktuellen Modell deutlich bessere Ergebnisse als das beste Modell mit einem vagen Satz.

9 Goals und lang laufende Aufgaben

Normalerweise arbeitet ein Agent einen Turn lang und wartet dann auf die nächste Nachricht. Für größere Aufgaben bedeutet das ständiges „mach weiter“. Goals ändern das: Man legt eine Zielbedingung fest, und der Agent arbeitet selbstständig weiter, bis sie erfüllt ist. Beide Werkzeuge haben dafür einen Befehl `/goal`, aber mit unterschiedlichen Eigenschaften.

9.1 Wie Goals funktionieren

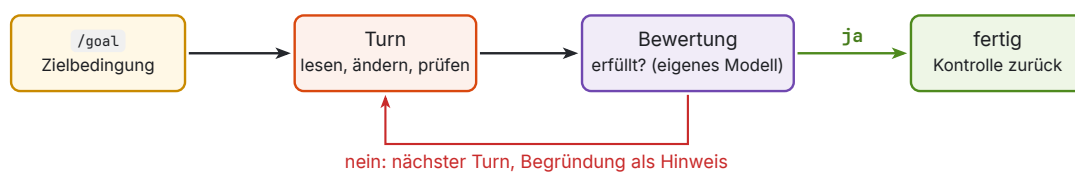


Abb. 10: Nach jedem Turn entscheidet eine separate Bewertung, ob die Bedingung erfüllt ist.

Claude Code (seit Version 2.1.139 [27]): `/goal <Bedingung>` setzt die Bedingung und startet sofort einen Turn. Nach jedem Turn schickt Claude Code die Bedingung und den bisherigen Verlauf an ein kleines, schnelles Modell (standardmäßig Haiku), das entscheidet, ob sie erfüllt ist. Technisch ist das ein sitzungsbezogener Stop-Hook. Pro Sitzung gibt es ein Goal. `/goal` ohne Argument zeigt Status, Turns, verbrauchte Token und die letzte Begründung der Bewertung, `/goal clear` beendet es, `/clear` ebenfalls. Mit `--resume` bleibt ein aktives Goal erhalten. Ein Goal ändert die Berechtigungen nicht: Damit es ohne Rückfragen läuft, braucht es einen passenden Berechtigungsmodus [12].

Codex: `/goal <Ziel>` hängt ein dauerhaftes Ziel an den Thread. Codex kennt die Zustände aktiv, pausiert, abgeschlossen und budgetbegrenzt, mit `/goal pause`, `/goal resume` und `/goal clear`. Ein Token-Budget begrenzt den Verbrauch. Eine Unterbrechung durch den Benutzer pausiert das Ziel. Das Modell selbst kann ein Ziel nur als abgeschlossen melden, Pause, Fortsetzen und Budget kontrolliert der Benutzer bzw. die Laufzeitumgebung. In der Desktop-App zeigt eine Fortschrittszeile über dem Eingabefeld das aktive Ziel [9], [28].

9.2 Ist ein Goal ein guter Ansatz?

Ja, für eine bestimmte Art von Aufgaben, und nein für viele andere. Entscheidend ist, ob sich das Ende **objektiv aus dem Verlauf feststellen** lässt.

Gut geeignet	Schlecht geeignet
Alle Aufrufer einer veralteten Funktion migrieren, bis der Build durchläuft	Architektur entwerfen oder zwischen Varianten entscheiden
Fehlschlagende Tests eines Moduls reparieren, bis alle grün sind	Unklare oder widersprüchliche Anforderungen umsetzen
Linter- oder Typfehler in einem Verzeichnis beseitigen	„Code verbessern“, „aufräumen“, „performanter machen“ ohne Messgröße
Akzeptanzkriterien einer bereits geplanten Anforderung als Tests abdecken und erfüllen	Sicherheitskritische Änderungen, die vor dem Zusammenführen ohnehin geprüft werden müssen, im Blindflug
Abhängigkeits-Upgrade, bis Tests und Typprüfung sauber sind	Explorative Aufgaben, bei denen das Ziel erst beim Arbeiten klar wird

Die Faustregel: **Ein Goal ersetzt das ständige „mach weiter“, nicht das Denken über die Aufgabe.** Die Arbeit vor dem Goal (Anforderung verstehen, Plan prüfen, Grenzen festlegen) bleibt beim Menschen. Das Goal übernimmt die mechanische, iterative Phase danach.

9.3 Die Grenzen kennen

- **Die Bewertung sieht nur den Verlauf.** Bei Claude Code beurteilt das Bewertungsmodell nur, was im Gespräch sichtbar ist. Behauptet der Agent „alle Tests grün“, ohne die Ausgabe zu zeigen, kann die Bewertung darauf hereinfallen [12]. Deshalb gehört in die Bedingung, dass der Nachweis sichtbar sein muss.
- **Goal-Gaming.** Ein Agent, der eine Bedingung „Tests grün“ um jeden Preis erreichen soll, kann Tests abschwächen oder überspringen (vgl. die Warnung vor Test-Gaming in [8]). Die Bedingung muss das ausschließen, und Testdateien sollten geschützt sein.
- **Verbrauch.** Jeder Turn kostet. Ohne Obergrenze kann ein Goal viel Nutzungsvolumen verbrauchen, ohne voranzukommen. Codex hat dafür Budgets [28], bei Claude Code formuliert man die Grenze in der Bedingung [12].
- **Festfahren.** Auch Anthropic empfiehlt für unbeaufsichtigte Läufe, nach zwei bis drei automatischen Fortsetzungen ohne Fortschritt abubrechen und den Lauf zu prüfen [11].

9.4 Gute und schlechte Zielbedingungen

Schlecht	Besser
<code>/goal Tags-Feature fertig</code>	<code>/goal Alle Akzeptanzkriterien von FR-012 haben je einen Test mit Markierung req("FR-012"); uv run pytest endet mit Exit-Code 0 und die Ausgabe ist im Verlauf sichtbar; keine bestehende Testdatei wurde geändert; höchstens 25 Turns</code>
<code>/goal Code ist sauber</code>	<code>/goal uv run ruff check app/ und uv run lint-imports laufen ohne Befund, Ausgabe sichtbar; Änderungen nur in app/; kein neues Paket in pyproject.toml</code>
<code>/goal Migration abgeschlossen</code>	<code>/goal grep -rn "alte_funktion" app/ findet nichts mehr; uv run pytest ist grün, Ausgabe sichtbar; höchstens 30 Turns</code>

Jede gute Bedingung hat drei Teile: einen **beobachtbaren Endzustand** (ein Befehl mit eindeutigem Ergebnis), einen **Schutz gegen Abkürzungen** (was nicht geändert werden darf) und eine **Obergrenze**.

TIPP

Wer nicht weiß, wie er eine Bedingung formulieren soll, kann das Modell bitten: „Formuliere eine /goal-Bedingung für diese Aufgabe mit beobachtbarem Endzustand, Schutz gegen Abkürzungen und Obergrenze.“ Das Ergebnis vor dem Setzen prüfen, denn eine zu weiche Bedingung ist genau die Schwachstelle, die das Goal ausnutzen wird.

9.5 Verwandte Mechanismen

Mechanismus	Wann statt <code>/goal</code>
<code>/loop</code> (Claude Code)	Wiederholung in Zeitabständen, etwa „prüfe alle 10 Minuten, ob der Deploy fertig ist“. Endet, wenn man es stoppt oder die Arbeit getan ist [12].
Eigener Stop-Hook	Wenn die Fertig-Prüfung nicht von einem Modell, sondern von einem Skript kommen soll, etwa „Stoppen erst erlaubt, wenn <code>pytest</code> Exit-Code 0 liefert“. Deterministischer als <code>/goal</code> [29].
Plan-Modus	Vor jedem Goal, um Vorgehen und Grenzen festzulegen.
<code>ultracode</code> / Ultra	Wenn die Aufgabe groß ist und sich sinnvoll auf parallele Teilagenten aufteilen lässt [2], [3].
Normale Sitzung	Für alles, bei dem man die Zwischenergebnisse ohnehin ansehen will.

Für Softwareentwicklung mit hohen Qualitätsansprüchen ist die Kombination aus **Plan-Modus, dann Goal mit harter Bedingung, dann Review** besonders wirksam: Der Mensch bestimmt Richtung und Grenzen, der Agent erledigt die Iteration, die Prüfungen entscheiden über „fertig“.

10 ADRs und Anforderungen als Leitplanken

Wer mit Coding-Agenten arbeitet, stößt schnell auf *Spec-Driven Development* (SDD): Vor dem Code entsteht eine Spezifikation, daraus ein Plan, daraus Aufgaben, und der Agent setzt sie um. Die Idee ist richtig, die üblichen Werkzeuge bringen aber pro Feature mehrere neue Dokumente mit. Dieses Kapitel zeigt, dass ein Projekt mit funktionalen und nicht-funktionalen Anforderungen als Markdown samt Akzeptanzkriterien und mit ADRs den Kern von SDD bereits besitzt, und wie man diese vorhandenen Artefakte so schärft, dass Agenten zuverlässig damit arbeiten.

10.1 Was Spec-Driven Development will

Bekannte Umsetzungen sind etwa GitHub Spec Kit (Spezifikation, Plan und Aufgabenliste pro Feature, dazu eine projektweite „Verfassung“ mit Grundregeln [30]) und AWS Kiro (Anforderungen in strukturierter Notation, Entwurf und Aufgaben pro Feature [31]). Die Werkzeuge entwickeln sich schnell, die Grundgedanken sind stabil:

1. **Anforderungen vor Code**, mit prüfbaren Akzeptanzkriterien.
2. **Ein Plan**, der Anforderungen mit Architekturentscheidungen verbindet, bevor Code entsteht.
3. **Rückverfolgbarkeit** von der Anforderung bis zur Umsetzung.
4. **Projektweite Regeln**, die jeder Agent kennt.

Die Kritik betrifft nicht die Idee, sondern den Preis: pro Feature drei zusätzliche Dateien, die gepflegt werden müssen, generierte Spezifikationen, die niemand liest, und Dokumente, die nach dem Merge vom Code abweichen. Genau das soll hier vermieden werden.

10.2 Was bereits vorhanden ist

SDD-Baustein	Vorhandenes Gegenstück	Neues Artefakt?
Spezifikation pro Feature	FR- und NFR-Dateien mit Akzeptanzkriterien	nein
Projektweite Regeln („Verfassung“)	CLAUDE.md / AGENTS.md plus die ADRs	nein
Entwurf, Plan	Plan-Modus im Gespräch; bei Architektur-Tragweite ein ADR	nein
Aufgabenliste	Aufgabenliste des Agenten während der Sitzung	nein
Rückverfolgbarkeit	IDs in Testmarkierungen und Commit-Nachrichten	nein

Die Optimierung besteht also nicht darin, etwas hinzuzufügen, sondern die vorhandenen Dokumente **agententauglich** zu machen: eindeutig, prüfbar, verlinkt.

10.3 Anforderungen agententauglich schreiben

Ein Agent liest Anforderungen wörtlich. Was nicht dasteht, füllt er mit Annahmen. Fünf Eigenschaften machen den Unterschied:

Eigenschaft	Umsetzung
Stabile ID	FR-012, NFR-004. Die ID ist der Anker für Tests, Commits und Reviews.
Prüfbare Kriterien	Jedes Akzeptanzkriterium lässt sich in genau einen Test übersetzen. „Schnell“ ist kein Kriterium, „p95 unter 200 ms bei 50 gleichzeitigen Anfragen“ schon.
Ausdrückliche Grenzen	Ein Abschnitt „Nicht Teil dieser Anforderung“. Er verhindert mehr Overengineering als jede Anweisung im Prompt.
Verweise	Betroffene ADRs und verwandte Anforderungen per ID.
Offene Fragen	Ungeklärtes wird als solches markiert, damit der Agent fragt statt erfindet.

docs/anforderungen/FR-012.md

FR-012 Notizen mit Tags versehen

Benutzer können eigene Notizen mit Tags versehen, um sie zu ordnen.

Akzeptanzkriterien

- AK1: Gegeben eine eigene Notiz, wenn der Benutzer PUT /notizen/{id}/tags mit ["uni", "privat"] sendet, dann enthält die Notiz genau diese Tags (Antwort 200).
- AK2: Tags werden kleingeschrieben gespeichert; "Uni" und "uni" sind derselbe Tag.
- AK3: Höchstens 10 Tags pro Notiz, je Tag 1 bis 30 Zeichen aus [a-z0-9-]; sonst Antwort 422 mit Problem Details.
- AK4: Für eine fremde oder nicht existierende Notiz antwortet die API mit 404.

Nicht Teil dieser Anforderung

- Suche oder Filterung nach Tags (geplant als FR-013)
- Umbenennen oder globales Löschen von Tags

Bezüge

- ADR-0003 Schichtenarchitektur, ADR-0007 Fehlerformat nach RFC 9457
- NFR-004 Antwortzeiten

Offene Fragen

- keine

Nicht-funktionale Anforderungen brauchen zusätzlich eine **Messmethode**, sonst sind sie für einen Agenten nicht prüfbar. Im Beispiel übernimmt sie ein Lasttest mit Locust [32]:

docs/anforderungen/NFR-004.md

NFR-004 Antwortzeiten der Notiz-Endpunkte**## Akzeptanzkriterien**

- AK1: p95 der Antwortzeit unter 200 ms für GET /notizen und PUT /notizen/{id}/tags bei 50 gleichzeitigen Benutzern und 10.000 Notizen in der Datenbank.

Messung

- Lasttest tests/last/test_notizen.py (locust), Ergebnis im CI-Artefakt.

10.4 ADRs agententauglich schreiben

Ein ADR hält eine Architekturentscheidung mit Kontext und Konsequenzen fest, etwa im Format von Michael Nygard [33] oder MADR [34]. Das Fehlerformat in ADR-0007 folgt RFC 9457 [35]. Für Agenten lohnen sich zwei kleine Schärfungen **innerhalb** der bestehenden Abschnitte, kein neues Dokument:

- Unter **Konsequenzen** konkret benennen, was daraus für die Umsetzung folgt: was erlaubt ist, was nicht.
- Eine Zeile **Prüfbar durch**: Welche automatische Prüfung stellt sicher, dass der Code der Entscheidung folgt?

docs/adr/0003-schichtenarchitektur.md

ADR-0003 Schichtenarchitektur für die API

Status: akzeptiert (2026-03-12)

Kontext

Endpunkte griffen direkt auf die Datenbank zu; Geschäftsregeln waren verstreut.

Entscheidung

Drei Schichten: app.api (HTTP) -> app.service (Regeln) -> app.repository (DB).
Abhängigkeiten nur von oben nach unten.

Konsequenzen

- Endpunkte rufen nur Services auf, nie Repositories oder SQLAlchemy direkt.
- Services kennen keine FastAPI-Typen (Request, Response, HTTPException).
- Neue Schichten oder Querschnittsmodule nur mit neuem ADR.

Prüfbar durch

- import-linter-Vertrag "Schichten gemäß ADR-0003" (pyproject.toml), läuft in der CI.

Die Zeile „Prüfbar durch“ verwandelt eine Absichtserklärung in eine Leitplanke. Ein ADR ohne automatische Prüfung wird von Agenten genauso oft übersehen wie von Menschen.

10.5 Der Ablauf

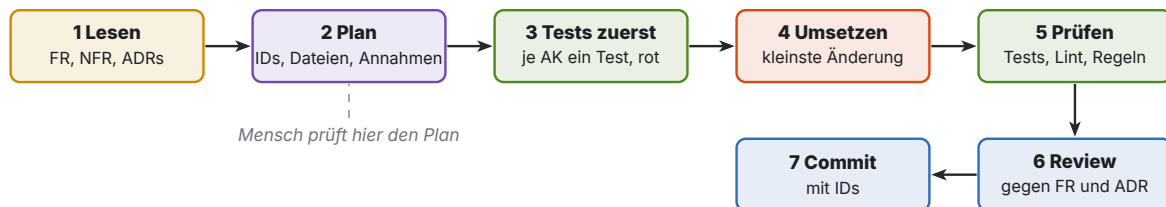


Abb. 11: Der Ablauf nutzt nur vorhandene Artefakte. Der Plan bleibt im Gespräch, Tests und Commit tragen die IDs.

- Lesen:** Der Agent liest die Anforderung und alle referenzierten ADRs und NFRs, bevor er etwas plant.
- Plan (Plan-Modus):** Er nennt die IDs, zählt die zu ändernden Dateien auf und legt Annahmen offen. Der Mensch prüft den Plan, hier ist Korrektur am billigsten.
- Tests zuerst:** Pro Akzeptanzkriterium ein Test, markiert mit der ID, zunächst rot. Die Tests sind die ausführbare Fassung der Anforderung.
- Umsetzen:** die kleinste Änderung, die alle Tests grün macht und die ADRs einhält. Diese Phase eignet sich für ein Goal (Kapitel 9).
- Prüfen:** Tests, Linter, Typprüfung, Architekturregeln, lokal und in der CI.
- Review:** durch das zweite Modell (Kapitel 6) und den Menschen, gegen Anforderung und ADRs.
- Commit** mit Verweis auf die IDs.

10.6 Rückverfolgbarkeit ohne Zusatzaufwand

Die IDs wandern in Artefakte, die ohnehin entstehen:

tests/test_tags.py (Auszug)

```

import pytest

@pytest.mark.req("FR-012", "AK3")
def test_mehr_als_zehn_tags_ergibt_422(client, eigene_notiz):
    antwort = client.put(f"/notizen/{eigene_notiz}/tags", json=[f"t{i}" for i in range(11)])
    assert antwort.status_code == 422
    assert antwort.headers["content-type"] == "application/problem+json"
  
```

pyproject.toml (Auszug)

```

[tool.pytest.ini_options]
markers = ["req(id, ak): verknüpft einen Test mit Anforderung und Akzeptanzkriterium"]
  
```

Eigene Markierungen sollten in der Konfiguration registriert sein, damit pytest bei Tippfehlern warnt (mit `--strict-markers` sogar abbricht) [36].

Tags für Notizen (FR-012)

Setzt AK1 bis AK4 um. Suche nach Tags ist nicht enthalten (FR-013).

Refs: FR-012

Damit lässt sich jede Frage mit einem Suchbefehl beantworten: Welche Tests decken FR-012 ab (`rg 'req\("FR-012"' tests/`)? In welchem Commit kam FR-012 (`git log --grep "FR-012"`)? Welches Akzeptanzkriterium hat noch keinen Test? Letzteres ist ein guter Prüfauftrag für das zweite Modell.

10.7 Architekturkonformität automatisch prüfen

Die Zeile „Prüfbar durch“ im ADR braucht eine Umsetzung. Für Python eignet sich *import-linter*, das Regeln über erlaubte Importe als Verträge prüft [37]:

pyproject.toml (Auszug)

```
[tool.importlinter]
root_package = "app"

[[tool.importlinter.contracts]]
name = "Schichten gemäß ADR-0003"
type = "layers"
layers = ["app.api", "app.service", "app.repository"]

[[tool.importlinter.contracts]]
name = "Services ohne FastAPI (ADR-0003)"
type = "forbidden"
source_modules = ["app.service"]
forbidden_modules = ["fastapi"]
```

`uv run lint-imports`

Der Befehl läuft lokal, im Goal als Teil der Bedingung und in der CI. Verstößt der Agent gegen einen ADR, erfährt er es sofort und mit Namen des Vertrags, statt erst im Review. Wer die Architektur zusätzlich als C4-Modell [38] pflegt, etwa in Structurizr DSL, kann Container- und Komponentengrenzen auf dieselbe Weise in Import-Verträge übersetzen und so prüfen, ob Code und Modell noch übereinstimmen.

10.8 Wann doch ein neues ADR entsteht

Stößt der Agent bei der Umsetzung auf eine Entscheidung mit Architektur-Tragweite (neue Abhängigkeit, neues Muster, Abweichung von einem bestehenden ADR), soll er sie nicht stillschweigend treffen. Die Regel in `CLAUDE.md` / `AGENTS.md`:

Architekturentscheidungen

Weicht eine Lösung von einem ADR ab oder erfordert sie eine neue Abhängigkeit, ein neues Muster oder eine neue Schicht: nicht umsetzen, sondern anhalten und einen ADR-Entwurf (Kontext, Optionen, Empfehlung) zur Entscheidung vorlegen.

So entstehen ADRs nur dort, wo tatsächlich entschieden wird, und der Agent wird vom Risiko zum Werkzeug: Er erkennt die Entscheidung, bereitet sie vor, und der Mensch trifft sie.

MERKE

Spec-Driven Development ohne neue Artefakte heißt: Anforderungen mit IDs, prüfbaren Kriterien und ausdrücklichen Grenzen, ADRs mit konkreten Konsequenzen und einer automatischen Prüfung, ein Plan im Gespräch statt in einer Datei, Tests und Commits als Träger der Rückverfolgbarkeit. Der Aufwand liegt in der Qualität der vorhandenen Dokumente, nicht in ihrer Anzahl.

11 Qualitätssicherung im Ablauf

Die vorigen Kapitel haben die Einzelteile beschrieben. Dieses Kapitel setzt sie zu einem Ablauf zusammen, in dem Qualität nicht vom Glück abhängt, sondern von Prüfungen, die jede Änderung durchlaufen muss.

11.1 Die Prüfpypamide

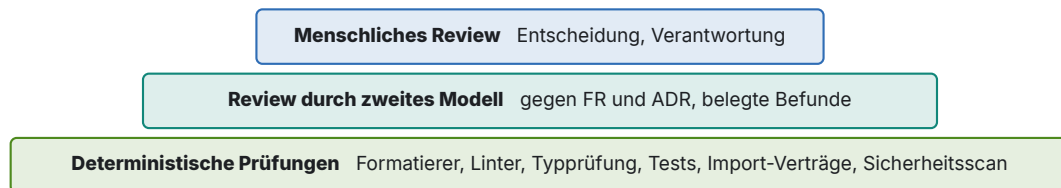


Abb. 12: Unten billig, schnell und bei jeder Änderung, oben teuer und gezielt. Was sich deterministisch prüfen lässt, prüft kein Modell und kein Mensch.

Die Reihenfolge ist entscheidend. Ein Modell-Review, das Formatierungsfehler meldet, verschwendet Aufmerksamkeit, und ein Mensch, der prüft, ob Tests laufen, ebenso. Jede Ebene kümmert sich um das, was nur sie leisten kann.

Prüfung	Befehl (ausgeführt mit <code>uv</code> [39])	findet
Formatierung, Stil	<code>uv run ruff format, ruff check</code> [40]	Stil, einfache Fehler, unsichere Muster (Regeln s)
Typen	<code>uv run mypy app/</code> [41] oder <code>pyright</code>	erfundene Attribute und Signaturen, falsche Typen
Tests	<code>uv run pytest</code>	Verhalten gegen Akzeptanzkriterien
Architektur	<code>uv run lint-imports</code> [37]	ADR-Verstöße
Abhängigkeiten	<code>uv run pip-audit</code> [42]	bekannte Schwachstellen

Die Typprüfung verdient besondere Erwähnung: Sie ist das günstigste Mittel gegen erfundene APIs, weil sie jede nicht existierende Methode und jeden falschen Parameter sofort meldet.

11.2 Taugen die Tests? Mutation Testing

Wenn Tests das Orakel sind (Kapitel 7), hängt alles an ihrer Qualität. Von Agenten geschriebene Tests haben eine typische Schwäche: Sie laufen grün, prüfen aber zu wenig, etwa nur den Normalfall oder nur, dass kein Fehler auftritt. Die Testabdeckung zeigt das nicht, denn sie misst nur, welcher Code ausgeführt wurde, nicht, ob ein Fehler aufgefallen wäre.

Mutation Testing beantwortet genau diese Frage [43]. Ein Werkzeug baut kleine, gezielte Fehler in den Code ein (*Mutanten*), etwa `>` statt `>=` oder `False` statt `True`, und führt die Tests aus. Schlägt ein Test fehl, ist der Mutant „getötet“, die Tests hätten den Fehler bemerkt. Laufen alle Tests weiter grün, hat der Mutant „überlebt“, und das ist eine Lücke in den Tests.

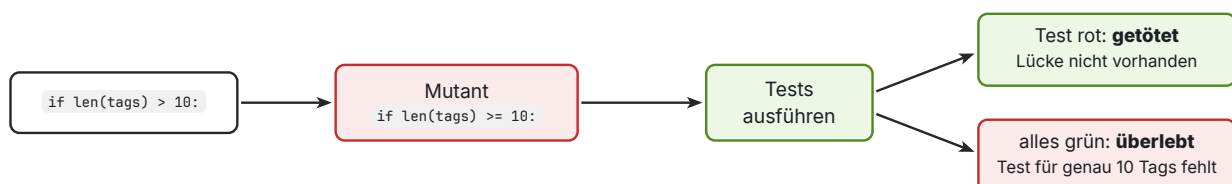


Abb. 13: Ein überlebender Mutant zeigt eine konkrete Lücke: Niemand prüft den Grenzfall mit genau 10 Tags.

Für Python ist *mutmut* das verbreitete Werkzeug [44]. Ab Version 3.7 heißen die Konfigurationsschlüssel `source_paths` und `pytest_add_cli_args_test_selection`, die älteren Namen `paths_to_mutate` und `tests_dir` gelten als veraltet:

pyproject.toml (Auszug)

```
[tool.mutmut]
source_paths = ["app/"]
pytest_add_cli_args_test_selection = ["tests/"]
```

```
uv add --dev mutmut
uv run mutmut run "app.notizen.tags*" # nur das geänderte Modul, sonst dauert es lange
uv run mutmut results                 # Überlebende Mutanten auflisten
uv run mutmut browse                  # interaktiv ansehen und gezielt neu testen
```

Die Ergebnisse legt *mutmut* im Verzeichnis `mutants/` ab, das in die `.gitignore` gehört. Das Werkzeug braucht ein System mit `fork`, also macOS oder Linux, unter Windows WSL [44].

Für die Arbeit mit Agenten ergibt sich ein einfaches Muster: Nach der Umsetzung laufen die Mutanten **nur für die geänderten Module**. Die überlebenden Mutanten gehen als Auftrag an den Agenten: „Ergänze Tests, sodass diese Mutanten getötet werden. Ändere keinen Produktivcode.“ Meta setzt ein verwandtes Verfahren im Industrieinsatz ein: Ein Sprachmodell erzeugt gezielt Tests, die bisher unentdeckte Fehler aufdecken [45]. Das ist eine der wenigen Aufgaben, bei der ein Goal ideal passt, weil das Ende eindeutig messbar ist. Eine vollständige Mutationsanalyse über das ganze Projekt dauert zu lange für jeden Commit, sie passt besser in einen nächtlichen Lauf der CI. Ein Wert von 100 Prozent ist dabei kein sinnvolles Ziel, manche Mutanten verändern das Verhalten gar nicht. Es geht darum, dass die Grenzfälle der Akzeptanzkriterien abgesichert sind.

11.3 Prüfungen automatisch auslösen: Hooks

Beide Werkzeuge können bei bestimmten Ereignissen Befehle ausführen. In Claude Code heißen sie *Hooks* und werden in `settings.json` definiert [29]. Ein Hook nach jedem Bearbeiten von Dateien formatiert und prüft sofort, sodass der Agent Fehler im selben Schritt sieht:

.claude/settings.json (Auszug)

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "uv run ruff format --quiet && uv run ruff check --quiet" }
        ]
      }
    ]
  }
}
```

Ein Stop-Hook kann darüber hinaus verhindern, dass der Agent eine Aufgabe beendet, solange eine Prüfung fehlschlägt. Das ist die deterministische Variante eines Goals (Kapitel 9) [12]. Codex bietet ebenfalls Hooks an, die in der Codex-Dokumentation unter *Hooks* beschrieben sind [46]. Hooks sind mächtig, weil sie ohne Rückfrage laufen: Nur Befehle eintragen, die man auch einem Kollegen ohne Nachfrage erlauben würde.

11.4 Berechtigungen und Sandbox

Qualität heißt auch, dass ein Agent nichts tun kann, was er nicht tun soll. Die Sicherheitsaspekte dazu vertieft Kapitel 12.

	Claude Code [26], [47]	Codex [21]
nur lesen und planen	Berechtigungsmodus <code>plan</code>	<code>/plan</code> , Sandbox <code>read-only</code>
Normalbetrieb	Standardmodus: fragt vor Änderungen und Befehlen, Freigaben über <code>permissions.allow</code>	Sandbox <code>workspace-write</code> mit Freigabe bei Bedarf
weitgehend selbstständig	Auto-Modus: ein Klassifikator entscheidet über Rückfragen	weniger Freigaben, gezielt pro Projekt
Schutz sensibler Dateien	<code>permissions.deny</code> , etwa <code>.env</code> , <code>secrets/</code>	Sandbox-Regeln, Netzwerkzugriff standardmäßig aus

Für unbeaufsichtigte Goals braucht es einen Modus, der nicht bei jedem Schritt fragt. Gerade dann gehören Tests und sensible Dateien geschützt und Netzwerkzugriff beschränkt.

11.5 Checkliste für das Review von KI-Änderungen

Frage
☐ Deckt jeder Test genau ein Akzeptanzkriterium ab, und hat jedes Kriterium einen Test?
☐ Enthält der Diff etwas, das in keiner Anforderung steht (Scope Creep)?
☐ Wurden bestehende Tests geändert? Wenn ja: war das Teil des Auftrags?
☐ Neue Abhängigkeiten, Konfigurationsoptionen, Abstraktionen: verlangt oder nur möglich?
☐ Hält die Änderung alle referenzierten ADRs ein (und laufen die Import-Verträge)?
☐ Fehlerbehandlung an Systemgrenzen, nicht verstreut über interne Aufrufe?
☐ Sind alle Aussagen der Zusammenfassung durch Befehlsausgaben belegt?
☐ Nennt die Zusammenfassung getroffene Annahmen und nicht umgesetzte Ideen?

11.6 Wissen, wann Schluss ist

Eine funktionierende, geprüfte Lösung weiter zu verfeinern, ist verlockend, weil ein Agent jede Verbesserung in Sekunden liefert. Aber jede Änderung ist ein neues Risiko und neuer Review-Aufwand. Wenn die Akzeptanzkriterien erfüllt, die Prüfungen grün und das Review abgeschlossen sind, ist die Aufgabe fertig. Weitere Runden für kosmetische Verbesserungen haben fast immer ein schlechtes Verhältnis von Nutzen zu Risiko. Wer merkt, dass die Ergebnisse von Runde zu Runde nur noch marginal besser werden, sollte das als Signal nehmen, aufzuhören, und nicht als Anlass für eine weitere Runde.

MERKE

Qualität mit Coding-Agenten entsteht aus vier Schichten: klare Anforderungen mit prüfbaren Kriterien, deterministische Prüfungen, die jede Änderung durchläuft, ein zweites Modell als unabhängiger Prüfer und ein Mensch, der entscheidet. Modell und Effort bestimmen, wie schnell man dorthin kommt, nicht ob.

12 Sicherheit beim Arbeiten mit Agenten

Ein Coding-Agent führt Befehle mit den Rechten dessen aus, der ihn gestartet hat. Er liest Dateien, Webseiten und Ausgaben von Werkzeugen und handelt danach. Das macht ihn nützlich und zugleich zu einem Angriffsziel: Wer beeinflussen kann, was der Agent liest, kann unter Umständen beeinflussen, was er tut [48].

12.1 Das Bedrohungsmodell

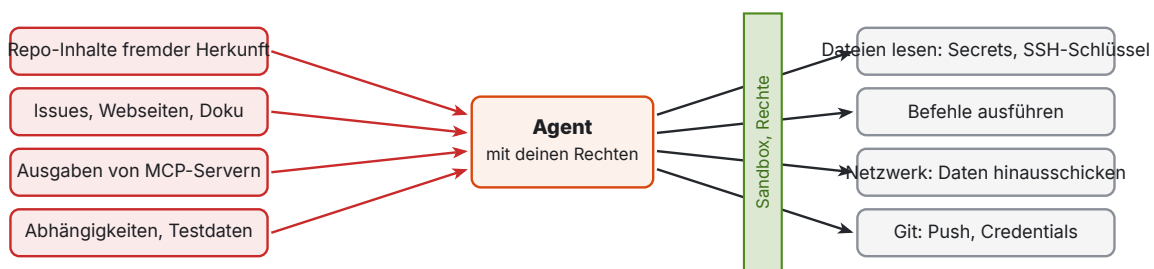


Abb. 14: Links, was der Agent liest und nicht kontrolliert wird. Rechts, was er tun kann. Dazwischen gehören die Leitplanken.

Die gefährlichste Kombination beschreibt Simon Willison als *lethal trifecta* [49]: Ein Agent hat **Zugriff auf private Daten**, liest **nicht vertrauenswürdige Inhalte** und kann **Daten nach außen schicken**. Sind alle drei gegeben, kann eine eingeschleuste Anweisung, etwa in einem Kommentar einer fremden Bibliothek oder in einer Webseite, den Agenten dazu bringen, Geheimnisse zu versenden. Das Ziel der Maßnahmen ist, mindestens eine der drei Bedingungen zu brechen.

Anthropic weist in der Sicherheitsdokumentation von Claude Code selbst darauf hin, dass Agenten durch Prompt Injection oder Modellfehler unbeabsichtigt handeln können, und empfiehlt mehrere Schutzschichten, statt sich auf eine zu verlassen [47].

12.2 Die Schutzschichten

Schicht	Claude Code [47]	Codex [21]
Sandbox	<code>/sandbox</code> : Isolation von Dateisystem und Netzwerk auf Betriebssystemebene (am Mac über Seatbelt [50])	Sandbox <code>workspace-write</code> bzw. <code>read-only</code> , Netzwerk standardmäßig aus
Berechtigungen	Manual-Modus fragt vor Änderungen und Befehlen, <code>permissions.deny</code> sperrt Dateien und Werkzeuge	Freigabe-Richtlinie (<code>approval_policy</code>), Regeln und Profile
Netzwerk	<code>curl</code> , <code>wget</code> werden nicht automatisch freigegeben, Netzwerkzugriffe im Manual-Modus nur nach Rückfrage	<code>network_access = false</code> in der Sandbox-Konfiguration
Auto-Modus	ein Klassifikator prüft Aktionen statt des Menschen; eigene Deny-Regeln gelten weiter	automatisches Review von Aktionen (<i>Auto-review</i>)

`.claude/settings.json` (Auszug, gehärtet)

```
{
  "permissions": {
    "deny": [
      "Read(/.env)", "Read(/.env.*)", "Read(/secrets/**)",
      "Read(/.ssh/**)", "Read(/.aws/**)",
```

```

    "Bash(curl:*)", "Bash(wget:*)", "Bash(git push:*)"
  ]
}
}

```

~/codex/config.toml (Auszug)

```

sandbox_mode = "workspace-write"
approval_policy = "on-request"

[sandbox_workspace_write]
network_access = false

```

ACHTUNG

Deny-Regeln passen auf den Befehl, wie er geschrieben ist. Ein Agent, der statt `curl` ein Python-Skript mit `urllib` ausführt, umgeht eine Regel für `curl`. Verlässlichen Schutz gegen Datenabfluss bietet deshalb nur die Netzwerkisolation der Sandbox, nicht die Liste verbotener Befehle [47].

12.3 Fremde Repositories

Projektdateien können Befehle auslösen: Hooks in `.claude/settings.json`, MCP-Server in `.mcp.json`, Anweisungen in `CLAUDE.md` und `AGENTS.md`. Eine 2025 bekannt gewordene Lücke in Claude Code (CVE-2025-59536) erlaubte, Code aus einem Projekt auszuführen, bevor der Vertrauensdialog beim ersten Start bestätigt war (behoben in Version 1.0.111) [51]. Claude Code fragt beim ersten Öffnen eines Verzeichnisses nach Vertrauen, auch bei neuen MCP-Servern [47]. Für fremde Repositories (Abgaben von Studierenden, Open-Source-Projekte, Beispielcode) gilt deshalb:

1. Zuerst die agentenbezogenen Dateien selbst lesen: `.claude/`, `.codex/`, `.mcp.json`, `CLAUDE.md`, `AGENTS.md`, Git-Hooks.
2. Den Agenten zunächst nur lesend arbeiten lassen (Plan-Modus bzw. Sandbox `read-only`).
3. Dem Vertrauensdialog nur zustimmen, wenn diese Prüfung unauffällig war.
4. Für Analysen von Schadcode oder CTF-Material eine Wegwerfumgebung verwenden (Container oder VM ohne Zugangsdaten).

12.4 Geheimnisse und Zugangsdaten

- **Keine Produktionszugänge in der Umgebung des Agenten.** Keine Deploy-Schlüssel, keine Cloud-Zugangsdaten mit Schreibrechten in der Shell, in der der Agent läuft.
- **Eigene, eingeschränkte Tokens,** etwa ein Gitea-Token nur mit Leserecht für das Repository, wenn der Agent überhaupt darauf zugreifen muss.
- **Push bleibt beim Menschen.** Der Agent committet in einen Branch, gepusht und gemergt wird nach dem Review. Geschützte Branches (Git-Handbuch, Kapitel 12 [20]) setzen das serverseitig durch.
- **Keine Geheimnisse in den Prompt kopieren.** Was im Kontext steht, kann in Logs, Zusammenfassungen und Commit-Nachrichten wieder auftauchen.

12.5 MCP-Server und Abhängigkeiten

MCP-Server erweitern den Agenten um Werkzeuge, und ihre Ausgaben landen ungeprüft im Kontext. Anthropic rät, eigene Server zu schreiben oder solche von vertrauenswürdigen Anbietern zu nutzen, und prüft sie selbst nicht auf Sicherheit [47]. Nur Server aus vertrauenswürdiger Quelle einsetzen, Versionen festlegen, ihnen minimale Rechte geben und ihre Ausgaben als nicht vertrauenswürdig betrachten.

Ein eigenes Risiko sind erfundene Paketnamen: Ein Modell schlägt ein Paket vor, das es nicht gibt, und ein Angreifer hat genau diesen Namen vorsorglich mit Schadcode registriert (*Slopsquatting* [52]). Messungen an 16 Modellen fanden erfundene Paketnamen bei mindestens 5,2 Prozent der Vorschläge kommerzieller und 21,7 Prozent der Vorschläge offener Modelle [53]. Die Regel „keine neuen Pakete ohne Zustimmung“ aus Kapitel 8 ist deshalb auch eine Sicherheitsregel. Vor der Zustimmung prüft man Name, Herkunft und Verbreitung des Pakets, danach läuft `pip-audit` in der CI.

12.6 Unbeaufsichtigte Läufe

Goals und Auto-Modus sparen Rückfragen, entfernen aber den Menschen als letzte Kontrolle. Für solche Läufe gilt eine strengere Konfiguration:

Maßnahme	
☐	Sandbox aktiv, Netzwerk nur zum Paketindex oder gar nicht
☐	eigener Worktree oder Container, keine Zugangsdaten in der Umgebung
☐	kein Push, kein Deployment, keine Datenbankzugriffe außerhalb der Testdatenbank
☐	Obergrenze an Turns bzw. Token-Budget
☐	Review des gesamten Diffs, bevor irgendetwas den Branch verlässt
☐	<code>bypassPermissions</code> bzw. vergleichbare Modi nur in Wegwerfumgebungen [26]

MERKE

Sicherheit mit Agenten folgt demselben Prinzip wie im Secure-API-Handbuch [5]: nichts, was von außen kommt, ist vertrauenswürdig, auch nicht Text, den der Agent liest. Die wirksamste Maßnahme ist, die Fähigkeiten des Agenten so zu begrenzen, dass eine erfolgreiche Manipulation keinen Schaden anrichten kann: kein Zugang zu Geheimnissen, kein freies Netzwerk, kein Push.

13 Ein Durchlauf von Anfang bis Ende

Dieses Kapitel spielt eine Anforderung vollständig durch: FR-012 „Notizen mit Tags versehen“ aus Kapitel 10, von der Anforderung bis zum Commit. Die Ausgaben der Werkzeuge sind gekürzt und vereinfacht, der Ablauf entspricht dem Vorgehen aus den vorigen Kapiteln.



Abb. 15: Die acht Schritte des Durchlaufs, Schritt 7 übernimmt Codex. Der Mensch greift an zwei Stellen ein: beim Plan und vor dem Commit.

13.1 Schritt 1: Sitzung in einem eigenen Worktree

```
cd ~/projekte/notizen
claude --worktree fr-012          # eigener Branch, eigenes Verzeichnis
```

Das Modell bleibt beim Standard Opus 5.5 mit Effort `medium`, das entspricht der Entscheidungstabelle für ein Feature mit klaren Akzeptanzkriterien (Kapitel 5). Mit `⌘ Tab` geht es in den Plan-Modus [26]. Die Option `--worktree` legt dafür ein eigenes Arbeitsverzeichnis unter `.claude/worktrees/` an [23].

13.2 Schritt 2: Plan anfordern und korrigieren

Setze FR-012 (docs/anforderungen/FR-012.md) um. Beachte die dort genannten ADRs.
 Erlaubte Pfade: app/notizen/, app/db/, tests/. Keine neuen Abhängigkeiten.
 Erst Plan (IDs, Dateien, Annahmen), dann Tests pro Akzeptanzkriterium, dann Umsetzung.
 Nicht Teil der Aufgabe: Suche nach Tags (FR-013).

Plan für FR-012 (AK1-AK4), beachtet ADR-0003 und ADR-0007

Dateien:

app/db/models.py	Tabelle tag, Zuordnungstabelle notiz_tag
app/repository/tags.py	neues TagRepository
app/service/tags.py	neuer TagService mit Cache für häufige Tags
app/api/notizen.py	Endpunkt PUT /notizen/{id}/tags
tests/test_tags.py	je ein Test für AK1-AK4
alembic/versions/...	Migration

Annahmen:

- Tags werden global geteilt, damit spätere Suche (FR-013) effizient ist.
- Ein Cache verbessert NFR-004.

Frage: Sollen Tags bei der Ausgabe ebenfalls kleingeschrieben sein? (AK2)

Der Plan zeigt genau die Muster aus Kapitel 8: eine eigene Tabelle „damit spätere Suche effizient ist“ (vorweggenommene Anforderung FR-013) und ein Cache ohne Messung. Hier ist die Korrektur am billigsten, ein Satz statt einer Stunde Code:

Keine eigene Tag-Tabelle und kein Cache: FR-013 ist nicht Teil der Aufgabe, und NFR-004 ist ohne Cache zu prüfen. Speichere Tags als Spalte tags (Text-Array) an der Notiz. Zu AK2: ja, auch die Ausgabe ist kleingeschrieben. Passe den Plan an.

Überarbeiteter Plan:

app/db/models.py Spalte tags (ARRAY(Text)) an Notiz

app/service/notizen.py	Funktion tags_setzen (Validierung AK2, AK3)
app/repository/notizen.py	Methode tags_speichern
app/api/notizen.py	Endpunkt PUT /notizen/{id}/tags
tests/test_tags.py	je ein Test für AK1-AK4
alembic/versions/...	Migration (Spalte hinzufügen)

Annahmen: keine offenen.

13.3 Schritt 3: Tests zuerst

Nach Freigabe des Plans schreibt der Agent die Tests, markiert mit den IDs, und zeigt, dass sie fehlschlagen:

```
$ uv run pytest tests/test_tags.py -q
FFFF [100%]
4 failed in 0.84s
```

Vier rote Tests, einer pro Akzeptanzkriterium: Die Anforderung liegt jetzt in ausführbarer Form vor.

13.4 Schritt 4: Umsetzung als Goal

Die Umsetzung ist mechanisch und hat ein messbares Ende, also passt ein Goal (Kapitel 9) [12]:

```
/goal Alle Tests in tests/test_tags.py und die übrigen Tests sind grün; uv run ruff
check ., uv run mypy app/ und uv run lint-imports laufen ohne Befund; die Ausgaben
sind im Verlauf sichtbar; tests/test_tags.py wird nicht mehr geändert; keine
Änderung außerhalb der erlaubten Pfade; höchstens 20 Turns.
```

```
/goal
  © Ziel erreicht nach 4 Turns
  Begründung: pytest 47 passed, ruff und mypy ohne Befund, lint-imports:
  2 contracts kept, 0 broken. Keine Änderung an tests/test_tags.py.
```

13.5 Schritt 5: Prüfen

```
$ git diff --stat main
alembic/versions/0012_notiz_tags.py | 22 ++++++++
app/api/notizen.py                  | 14 ++++++
app/db/models.py                    |  2 +
app/repository/notizen.py            |  9 +++++
app/service/notizen.py               | 21 ++++++++
tests/test_tags.py                  | 48 ++++++++
6 files changed, 116 insertions(+)
```

Sechs Dateien, alle innerhalb der erlaubten Pfade und entlang der Schichten aus ADR-0003, keine Änderung an `pyproject.toml`. Der Diff ist so groß, wie die Anforderung es erwarten lässt.

13.6 Schritt 6: Taugen die Tests?

```
$ uv run mutmut run "app.service.notizen.x_tags_setzen*"
...
$ uv run mutmut results
app.service.notizen.x_tags_setzen__mutmut_4: survived
app.service.notizen.x_tags_setzen__mutmut_9: survived
```

Zwei überlebende Mutanten [44]: `> 10` wurde zu `>= 10` verändert, und die Längengrenze 30 wurde zu 31. Beide Grenzfälle fehlen in den Tests. Auftrag an den Agenten:

```
Ergänze Tests, sodass die überlebenden Mutanten 4 und 9 getötet werden: genau 10
Tags sind erlaubt, ein Tag mit genau 30 Zeichen ist erlaubt, mit 31 nicht.
Ändere keinen Produktivcode.
```

Danach sind beide Mutanten getötet. Die Tests prüfen jetzt die Grenzen von AK3, nicht nur einen Fall weit darüber.

13.7 Schritt 7: Review durch Codex

In einem zweiten Terminal läuft Codex nur lesend über den Branch:

```
cd ~/projekte/notizen/.claude/worktrees/fr-012
codex exec -m gpt-6.1-sol -c model_reasoning_effort="high" -s read-only \
  "Prüfe git diff main...HEAD gegen docs/anforderungen/FR-012.md und die dort
  genannten ADRs. Nur belegte Befunde mit Datei, Zeile, ID und Fehlerfall.
  Keine Stilfragen. Ändere keine Dateien."
```

Befund 1 (FR-012 AK4): tests/test_tags.py prüft 404 nur für eine fremde Notiz,
nicht für eine nicht existierende ID. AK4 verlangt beides.
Befund 2 (ADR-0007): app/api/notizen.py:88 gibt bei ungültigen Tags ein JSON mit
"detail" zurück, nicht application/problem+json.
Hinweis: Die Variable t in app/service/notizen.py:52 könnte sprechender heißen.

Befund 1 und 2 sind belegt und betreffen Anforderung und ADR (ADR-0007 verlangt das Fehlerformat `application/problem+json` nach RFC 9457 [35]), sie gehen an Claude zurück. Der Hinweis zur Benennung ist eine Stilfrage ohne Fehler und wird nach der Regel aus Kapitel 6 verworfen. Nach der Korrektur laufen Tests und Prüfungen erneut grün. Damit ist der Durchgang beendet, es gibt keine zweite Review-Runde zwischen den Modellen.

13.8 Schritt 8: Menschliches Review und Commit

Das Review folgt der Checkliste aus Kapitel 11: Jedes Akzeptanzkriterium hat einen Test, der Diff enthält nichts außerhalb von FR-012, keine bestehenden Tests wurden geändert, keine neuen Pakete, ADR-0003 und ADR-0007 sind eingehalten. Dann der Commit:

Tags für Notizen (FR-012)

Setzt AK1 bis AK4 um. Tags als Spalte an der Notiz; Suche nach Tags ist
nicht enthalten (FR-013).

Refs: FR-012

13.9 Was der Durchlauf zeigt

Schritt	Was es gebracht hat
Plan-Modus	Zwei Fälle von Overengineering (eigene Tabelle für eine künftige Anforderung, Cache ohne Messung) für einen Satz Korrektur entfernt.
Tests zuerst	Die Anforderung wurde ausführbar, bevor Code entstand. Das Goal hatte ein eindeutiges Ende.
Goal	Die mechanische Umsetzung lief ohne ständiges „mach weiter“.
Mutation Testing	Zwei Grenzfälle gefunden, die grüne Tests nicht abgedeckt hatten.
Codex-Review	Eine Lücke bei AK4 und ein ADR-Verstoß gefunden, die dem Umsetzer entgangen waren.
Mensch	Richtung am Anfang, Entscheidung am Ende. Dazwischen keine Kleinarbeit.

Kein Schritt dieses Ablaufs hat ein neues Dokument erzeugt. Die Anforderung und die ADRs existierten vorher, der Plan blieb im Gespräch, und die Rückverfolgbarkeit steckt in Testmarkierungen und Commit-Nachricht.

14 System-Prompt und Inferenz-Einstellungen

Bisher ging es um Modell und Effort als Regler. Darunter liegt eine Ebene, die man bei Coding-Werkzeugen selten sieht, die aber erklärt, warum sie sich so verhalten: Was genau geht bei jeder Anfrage an das Modell, und mit welchen Einstellungen wird die Antwort erzeugt?

14.1 Was bei jeder Anfrage an das Modell geht

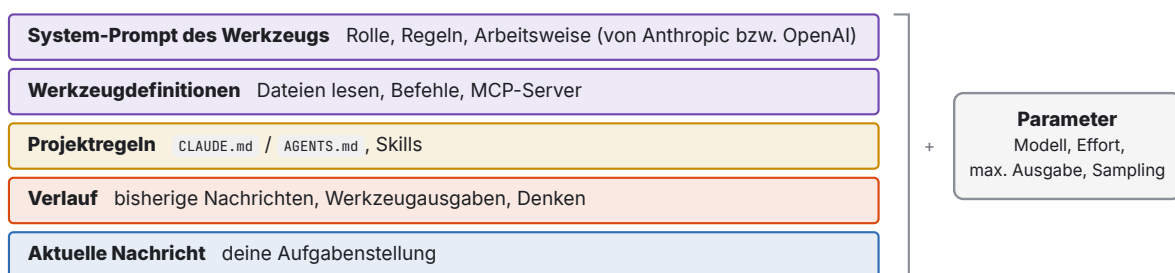


Abb. 16: Aufbau einer Anfrage an das Modell, von oben (stabil) nach unten (wechselnd). Stabile Teile vorne machen Prompt-Caching möglich.

Die Reihenfolge ist kein Zufall. Alles, was sich zwischen Anfragen nicht ändert, steht vorne und kann vom Anbieter zwischengespeichert werden (*Prompt-Caching* [17]). Deshalb kosten lange Sitzungen weniger als ihre Token-Zahl vermuten lässt, und deshalb machen Änderungen an den vorderen Schichten mitten in der Sitzung den Cache ungültig [17].

14.2 Der System-Prompt

Der System-Prompt legt fest, wer das Modell in diesem Gespräch ist, welche Regeln gelten und wie es arbeiten soll. Er hat mehr Gewicht als eine normale Nachricht und gilt für die ganze Sitzung.

Umgebung	Wie man ihn beeinflusst
Claude Code	Das Werkzeug bringt einen umfangreichen eigenen System-Prompt mit. Der vorgesehene Weg für eigene Regeln ist <code>CLAUDE.md</code> . Für Skripte gibt es <code>--append-system-prompt "..."</code> (ergänzt) und <code>--system-prompt "..."</code> (ersetzt) [54]. Ersetzen entfernt die Arbeitsanweisungen des Werkzeugs und ist für normale Entwicklung fast nie sinnvoll.
Codex	Eigene Basisanweisungen des Werkzeugs; eigene Regeln gehören in <code>AGENTS.md</code> , die Codex in jede Sitzung einbindet [55].
Chat-Apps	Projekt-Anweisungen (Claude-Projekte) bzw. Anpassungen und Projekte in ChatGPT.
API	Claude: Feld <code>system</code> . OpenAI Responses API: Feld <code>instructions</code> bzw. eine Nachricht mit Rolle <code>developer</code> .

Was einen guten System-Prompt ausmacht, deckt sich mit den Regeln für `CLAUDE.md` und `AGENTS.md` aus Kapitel 8:

- **Rolle und Kontext** knapp: für wen, in welchem Projekt, mit welchem Ziel.
- **Regeln als Verhalten formulieren**, nicht als Gesinnung: „Ändere nur, was verlangt ist“ statt „Sei sorgfältig“.
- **Ausgabeformat** festlegen, wo es gebraucht wird.
- **Nicht übersteuern**. Anthropic weist darauf hin, dass aktuelle Modelle auf Anweisungen stärker reagieren als frühere [8]. Formulierungen wie „Nutze im Zweifel immer Werkzeug X“ führen heute zu übermäßigem Einsatz. Großbuchstaben und „MUSS“ sind selten nötig.

- **Widersprüche vermeiden.** Ein System-Prompt, der „arbeite selbstständig“ und „frage bei jeder Unklarheit“ enthält, erzeugt unvorhersehbares Verhalten.
- **Eingefügte Texte kennzeichnen.** Für Anwendungen, in denen Nutzer fremde Texte einfügen, empfiehlt Anthropic bei Opus 5.5, diese mit Markierungen zu umschließen und im System-Prompt zu erklären, dass Anweisungen darin nicht zu befolgen sind [11].

14.3 Inferenz-Einstellungen: wie aus Wahrscheinlichkeiten Text wird

Ein Sprachmodell berechnet für jedes nächste Token eine Wahrscheinlichkeitsverteilung über sein gesamtes Vokabular. Die Inferenz-Einstellungen bestimmen, wie aus dieser Verteilung ausgewählt wird (*Sampling*).

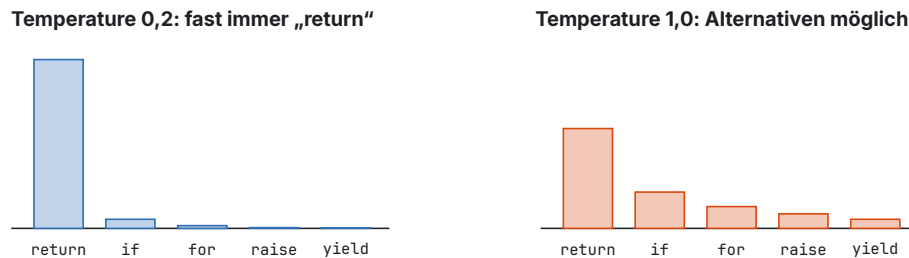


Abb. 17: Schematisch: Niedrige Temperature schärft die Verteilung, hohe flacht sie ab. Die Werte sind illustrativ.

Parameter	Wirkung
<code>temperature</code>	Schärft (klein) oder glättet (groß) die Verteilung. 0 heißt: immer das wahrscheinlichste Token (<i>greedy</i>).
<code>top_p</code>	<i>Nucleus Sampling</i> [56]: nur die wahrscheinlichsten Token, bis ihre Summe <code>p</code> erreicht (etwa 0,9), der Rest fällt weg.
<code>top_k</code>	nur die <code>k</code> wahrscheinlichsten Token berücksichtigen.
<code>min_p</code>	nur Token, deren Wahrscheinlichkeit mindestens <code>min_p</code> mal die des besten Tokens beträgt [57]. Verbreitet bei lokalen Modellen.
Wiederholungsstrafen	<code>repeat_penalty</code> , <code>presence_penalty</code> , <code>frequency_penalty</code> senken die Wahrscheinlichkeit bereits verwendeter Token.
<code>seed</code>	Startwert des Zufallsgenerators, für reproduzierbare Läufe.
<code>max_tokens</code>	Obergrenze der Ausgabe. Bei Reasoning-Modellen zählt das Denken mit.
Stop-Sequenzen	Zeichenfolgen, bei denen die Ausgabe endet.

14.4 Was davon bei Claude und Codex einstellbar ist

Hier liegt die wichtigste Änderung der letzten Modellgenerationen: **Bei den aktuellen Frontier-Modellen ist Sampling kein Regler mehr.**

Modell / Umgebung	Sampling-Parameter
Claude Opus 4.7 und neuer (auch Opus 5.5), Sonnet 5	<code>temperature</code> , <code>top_p</code> und <code>top_k</code> mit anderem als dem Standardwert führen zu einem Fehler 400. Anthropic nennt Effort und Prompting als die Wege, das Verhalten zu steuern [15].
OpenAI GPT-5.6	Nur der Standardwert von <code>temperature</code> (1) wird akzeptiert.
OpenAI GPT-6 Astra	<code>temperature</code> und <code>top_p</code> werden abgelehnt [58].
OpenAI GPT-6 Sol, Luna	Nur ohne Reasoning (Effort <code>none</code>) zulässig, sonst abgelehnt [58], [59].
Claude Code, Codex, Desktop-Apps	keine Sampling-Einstellungen; die Werkzeuge setzen ihre Parameter selbst.

Der Grund: Reasoning-Modelle sind auf ein bestimmtes Sampling-Verhalten hin trainiert und abgestimmt. Wer die Streuung verringern will, erreicht das bei diesen Modellen über klare Anweisungen, strukturierte

Ausgaben (vorgegebene JSON-Schemata) und Prüfungen, nicht über die Temperature. Für Code gilt ohnehin: Reproduzierbarkeit entsteht durch Tests, nicht durch deterministisches Sampling.

ACHTUNG

Bibliotheken und Werkzeuge, die für ältere Modelle eine Temperature mitschicken, erzeugen bei den aktuellen Modellen Fehler. Wer eigene Anwendungen gegen die APIs baut, lässt Sampling-Parameter bei Claude Opus 4.7+ und bei OpenAI-Reasoning-Modellen einfach weg.

14.5 Kontextlänge

Die Kontextlänge ist die Obergrenze für alles aus der Abbildung oben zusammen, einschließlich Ausgabe und Denken. Drei Zahlen werden oft verwechselt:

Größe	Bedeutung
Kontextfenster des Modells	Was das Modell technisch verarbeiten kann, bei den aktuellen Claude-Modellen eine Million Token [1].
Wirksames Fenster im Werkzeug	Was das Werkzeug tatsächlich nutzt. Bei Codex lag es laut Berichten von Nutzern zeitweise deutlich unter dem Wert der API-Dokumentation. Claude Code verdichtet bei etwa 967.000 Token [2].
Maximale Ausgabe	Obergrenze einer einzelnen Antwort, einschließlich Denken.

Ein großes Fenster ist kein Freibrief. Jedes Token im Verlauf wird bei jeder Anfrage erneut verarbeitet, das kostet Nutzungsvolumen, und mit wachsendem Verlauf steigt das Risiko, dass frühe Details übersehen oder durch spätere überlagert werden [60]. Die Regel aus Kapitel 1 gilt deshalb unabhängig von der Fenstergröße: eine Aufgabe pro Sitzung, dauerhafte Informationen in Dateien statt im Verlauf.

15 Lokale Modelle: Formate, Speicher und Sampling

Für manche Aufgaben ist ein lokal betriebenes Modell die bessere Wahl: wenn Daten das Haus nicht verlassen dürfen (etwa Freitextantworten von Kunden), wenn große Mengen gleichartiger Aufgaben anfallen oder wenn Kosten pro Anfrage eine Rolle spielen. Für agentisches Coding reichen lokale Modelle an Claude und Codex nicht heran, für Extraktion, Klassifikation, Zusammenfassung und qualitative Codierung aber oft sehr wohl. Hier kommen genau die Einstellungen ins Spiel, die bei den Frontier-Modellen gesperrt sind.

15.1 Die Formate

Ein Modell besteht aus Gewichten (den gelernten Zahlen), einem Tokenizer, einer Konfiguration und einer Vorlage für das Chat-Format. Die Formate unterscheiden sich darin, wie diese Teile verpackt sind und welche Laufzeitumgebung sie liest.

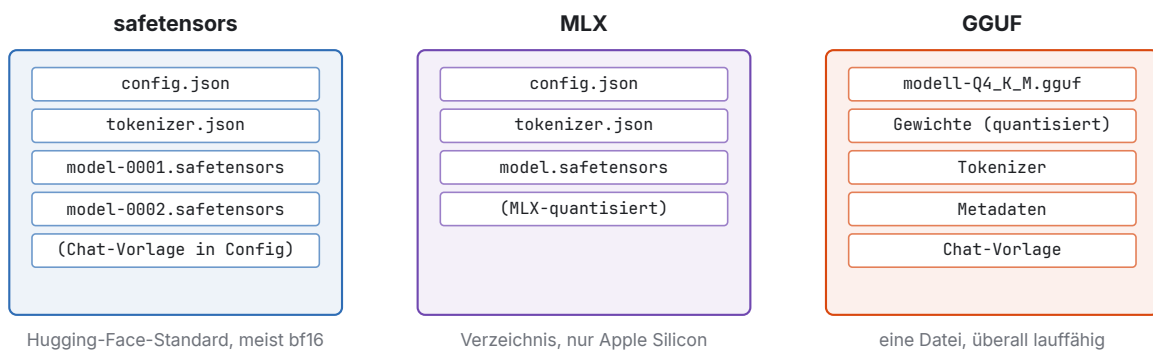


Abb. 18: safetensors und MLX sind Verzeichnisse mit mehreren Dateien, GGUF packt alles in eine einzige Datei.

	safetensors	MLX	GGUF
Was es ist	Standardformat für Gewichte auf Hugging Face, sicher ladbar (kein ausführbarer Code) [61]	Modellverzeichnis für Apples Framework MLX, Gewichte als safetensors in MLX-Quantisierung [62]	Container von llama.cpp mit Gewichten, Tokenizer, Metadaten und Chat-Vorlage in einer Datei [63], [64]
Laufzeit	transformers, vLLM, TGI	mlx-lm, LM Studio, Ollama (MLX-Backend)	llama.cpp, Ollama, LM Studio
Plattform	vor allem NVIDIA-GPUs	nur Apple Silicon	Mac, Linux, Windows, CPU, CUDA, Metal
Quantisierung	meist volle Genauigkeit; GPTQ und AWQ speichern quantisierte Gewichte ebenfalls als safetensors	3 bis 8 Bit	2 bis 8 Bit in vielen Varianten (K-Quants wie <code>Q4_K_M</code>)
Stärke	Ausgangspunkt für alles andere, Feinabstimmung, Server-Betrieb	höchste Geschwindigkeit auf dem Mac, nutzt den gemeinsamen Speicher direkt	Portabilität, riesiges Angebot, eine Datei

Auf einem Mac mit Apple Silicon ist MLX nach übereinstimmenden Messungen verschiedener Quellen spürbar schneller als GGUF bei gleicher Quantisierungsstufe, die genannten Werte reichen je nach Modell und Werkzeug von etwa 15 bis 50 Prozent [65], [66]. LM Studio nutzt MLX automatisch, wenn eine MLX-Fassung eines Modells existiert [67], Ollama hat seit Version 0.19 (Frühjahr 2026) ein MLX-Backend für Apple Silicon [66]. Umgekehrt halten die gemischten K-Quants von GGUF bei 4 Bit die Qualität teils etwas

besser als eine einfache 4-Bit-Quantisierung in MLX. Weitere Namen, die man antrifft: *GPTQ* [68] und *AWQ* [69] sind Quantisierungsverfahren (gespeichert als *safetensors*, für NVIDIA-GPUs), *EXL2* ist das Format von *ExLlamaV2*, ebenfalls für NVIDIA.

TIPP

Für den Mac gilt als Faustregel: MLX, wenn es eine Fassung von `mlx-community` oder dem Hersteller gibt, sonst GGUF in `q4_K_M` oder `q5_K_M`. Wer dasselbe Modell auch auf einem Linux-Server betreiben will, ist mit GGUF auf beiden Seiten einheitlicher. Konvertierungen von GGUF nach MLX sind möglich, übernehmen aber die Quantisierungsfehler der GGUF-Datei. Besser konvertiert man aus den ursprünglichen *safetensors*.

15.2 Quantisierung und Speicherbedarf

Quantisierung speichert Gewichte mit weniger Bits: statt 16 Bit pro Gewicht etwa 8 oder 4. Das spart Speicher und beschleunigt die Ausgabe, kostet aber etwas Qualität. 4 Bit gilt als üblicher Kompromiss, 8 Bit ist von voller Genauigkeit kaum zu unterscheiden, unter 4 Bit sinkt die Qualität deutlich.

Der Speicherbedarf setzt sich aus zwei Teilen zusammen:

Teil	Näherung
Gewichte	Parameterzahl × Bits ÷ 8, plus etwa 10 Prozent für Verwaltung
KV-Cache	wächst linear mit der Kontextlänge: 2 × Schichten × KV-Köpfe × Kopfdimension × Token × Bytes pro Wert

Modellgröße	4 Bit	8 Bit	Hinweis
8 Mrd.	ca. 5 GB	ca. 9 GB	läuft auf fast jedem Mac
32 Mrd.	ca. 18 GB	ca. 35 GB	ab 32 GB Arbeitsspeicher sinnvoll
122 Mrd.	ca. 65 GB	ca. 130 GB	ab 96 bis 128 GB
397 Mrd. (MoE, 17 Mrd. aktiv)	ca. 210 GB	ca. 420 GB	4 Bit braucht 256 GB

Beispielrechnung KV-Cache (der Cache wächst je Anfrage dynamisch mit der Kontextlänge [70]): Ein typisches 32-Milliarden-Modell mit 64 Schichten, 8 KV-Köpfen und Kopfdimension 128 braucht in 16 Bit pro Token $2 \times 64 \times 8 \times 128 \times 2$ Byte, also 256 KB. Bei 32.000 Token Kontext sind das 8 GB zusätzlich zu den Gewichten, bei 128.000 Token 32 GB. Ein 8-Bit-KV-Cache halbiert das bei kaum messbarem Qualitätsverlust.

MERKE

Mixture-of-Experts-Modelle (MoE [71], erkennbar an Namen wie `397B-A17B`) aktivieren pro Token nur einen Teil ihrer Parameter, hier 17 von 397 Milliarden. Das macht sie schnell wie ein 17-Milliarden-Modell. Im Speicher liegen müssen aber **alle** Parameter. Für die Speicherplanung zählt die Gesamtzahl, für die Geschwindigkeit die aktive.

Auf dem Mac darf die GPU standardmäßig nicht den gesamten gemeinsamen Speicher nutzen. Die Grenze lässt sich mit `sudo sysctl iogpu.wired_limit_mb=<Wert>` anheben (bis zum nächsten Neustart). Für das Betriebssystem und andere Programme muss trotzdem genug frei bleiben, etwa 8 bis 16 GB.

15.3 Kontextlänge lokal einstellen

Bei lokalen Modellen ist die Kontextlänge eine bewusste Einstellung, und hier lauert die häufigste Falle: **Ollama verwendet standardmäßig ein Fenster von 4.096 Token** [72], auch wenn das Modell viel mehr unterstützt. Längere Eingaben werden gekürzt, ohne dass es auffällt. Für die Codierung langer Texte oder das Lesen ganzer Dokumente muss das Fenster ausdrücklich vergrößert werden:

```
OLLAMA_CONTEXT_LENGTH=32768 ollama serve # Standard für alle Modelle
OLLAMA_FLASH_ATTENTION=1 OLLAMA_KV_CACHE_TYPE=q8_0 ollama serve # KV-Cache halbieren
```

In der API setzt man pro Anfrage `"options": {"num_ctx": 32768}`. Zu beachten: Mit `OLLAMA_NUM_PARALLEL` verarbeitet Ollama mehrere Anfragen gleichzeitig, der Speicherbedarf für den Kontext vervielfacht sich entsprechend [72]. In LM Studio stellt man die Kontextlänge beim Laden des Modells ein. Mehr als das Modell im Training gelernt hat (steht in der Model Card), bringt nichts oder verschlechtert die Qualität.

15.4 Sampling für lokale Modelle

Hier sind Temperature und Co. echte Stellschrauben. Zwei Regeln haben Vorrang vor jeder Tabelle:

1. **Die Model Card zuerst.** Viele Hersteller geben empfohlene Werte an. Qwen etwa empfiehlt für Qwen3 im Denkmodus `temperature` 0,6, `top_p` 0,95, `top_k` 20 und `min_p` 0, ohne Denkmodus 0,7 und 0,8, und rät ausdrücklich von *greedy decoding* (Temperature 0) im Denkmodus ab, weil es zu Endlosschleifen und Wiederholungen führt [73], [74].
2. **Reasoning-Modelle nicht auf 0 setzen.** Was bei älteren Modellen für „präzise“ stand, kann bei Modellen mit Denkphase das Gegenteil bewirken.

Aufgabe	Startwerte	Begründung
Extraktion, Klassifikation, Codierung	<code>temperature</code> 0,1 bis 0,3, <code>top_p</code> 0,9	wenig Streuung, gleiche Eingabe soll gleiche Kategorie ergeben
Code ohne Denkmodus	<code>temperature</code> 0,2, <code>top_p</code> 0,95	präzise, aber nicht starr
Modelle mit Denkmodus	Werte der Model Card	auf diese Werte hin trainiert
Formulieren, Zusammenfassen	<code>temperature</code> 0,6 bis 0,8	natürlichere Sprache

Für Auswertungen, die nachvollziehbar sein müssen, etwa die qualitative Codierung von Freitextantworten, reicht eine niedrige Temperature allein nicht. Auch mit `seed` und Temperature 0 sind Läufe nicht garantiert bitgleich, weil parallele Berechnung und Rundung auf der GPU leicht variieren. Nachvollziehbarkeit entsteht deshalb durch:

- **festgehaltene Konfiguration:** Modelldatei samt Prüfsumme, Quantisierung, Laufzeitversion, System-Prompt, alle Sampling-Parameter, Kontextlänge;
- **strukturierte Ausgabe:** ein festes Kategorienschema, das Ollama, LM Studio und llama.cpp über JSON-Schemata bzw. Grammatiken erzwingen können;
- **Konsistenzprüfung:** dieselben Texte mehrfach codieren lassen und die Übereinstimmung zwischen den Läufen messen, wie zwischen zwei menschlichen Codierern (etwa mit Krippendorffs Alpha [75]).

15.5 Chat-Vorlage und System-Prompt lokal

Jedes Modell erwartet seine Eingabe in einem bestimmten Format mit Sondertoken für System, Benutzer und Antwort. Diese *Chat-Vorlage* steckt in GGUF-Dateien in den Metadaten, bei safetensors und MLX in der Tokenizer-Konfiguration. Ist sie falsch oder fehlt sie, ignoriert das Modell den System-Prompt, antwortet mit seltsamen Token oder hört nicht auf zu schreiben. Seriöse Quellen (offizielle Fassungen, `mlx-community`, bekannte Quantisierer) liefern korrekte Vorlagen mit, bei selbst konvertierten Modellen ist das die erste Fehlerquelle.

In Ollama bündelt eine *Modelfile* Modell, Parameter und System-Prompt zu einer wiederverwendbaren Konfiguration, die sich versionieren lässt:

Modelfile

```
FROM ./modell-Q4_K_M.gguf
PARAMETER num_ctx 32768
PARAMETER temperature 0.2
PARAMETER top_p 0.9
PARAMETER seed 42
SYSTEM ""
Du codierst Freitextantworten von Kunden nach dem Codebuch unten.
Antworte ausschließlich mit JSON nach dem vorgegebenen Schema.
Wenn keine Kategorie passt, verwende "sonstiges" und begründe in einem Satz.
""
```

```
ollama create codierer -f Modelfile  
ollama run codierer
```

15.6 Lokale Modelle in Claude Code und Codex

Beide Werkzeuge lassen sich technisch mit lokalen Modellen verbinden: Codex bringt dafür einen eigenen Modus (`--oss`) für Ollama und LM Studio mit [9], Claude Code kann über `ANTHROPIC_BASE_URL` auf einen Server mit Anthropic-kompatibler Schnittstelle zeigen, wie ihn aktuelle Versionen von LM Studio anbieten [76]. Für agentisches Coding im Sinne dieses Buchs ist das derzeit aber keine gute Wahl. Die Werkzeuge sind auf die Fähigkeiten der Frontier-Modelle abgestimmt, und die Qualitätsunterschiede bei langen, werkzeuglastigen Aufgaben sind groß. Der sinnvolle Einsatz lokaler Modelle liegt bei klar umrissenen Aufgaben mit hohem Volumen und bei Daten, die das Haus nicht verlassen dürfen.

A Vorlagen

A.1 Eine Datei für beide Werkzeuge

Codex liest `AGENTS.md` [55], Claude Code liest `CLAUDE.md`. Damit die Regeln nicht doppelt gepflegt werden, steht der gemeinsame Inhalt in `AGENTS.md`, und `CLAUDE.md` bindet ihn per Import ein und ergänzt nur Claude-spezifisches [54]. Die Datei soll kurz bleiben (Anthropic empfiehlt unter 200 Zeilen [54]): Regeln und Befehle, keine Architekturbeschreibung (die steht in den ADRs).

AGENTS.md

```
# notizen - Regeln für Coding-Agenten

## Projekt
FastAPI + PostgreSQL, Python 3.13, Abhängigkeiten mit uv.
Anforderungen: docs/anforderungen/ (FR-xxx, NFR-xxx). Architektur: docs/adr/.

## Befehle
- Tests:          uv run pytest
- Lint/Format:    uv run ruff check . && uv run ruff format --check .
- Typen:          uv run mypy app/
- Architektur:    uv run lint-imports

## Vorgehen
- Lies vor jeder Umsetzung die betroffene Anforderung und alle dort genannten ADRs.
- Nenne im Plan die IDs, die zu ändernden Dateien und deine Annahmen.
- Schreibe pro Akzeptanzkriterium einen Test mit @pytest.mark.req("FR-xxx", "AKn").
- Setze die kleinste Änderung um, die alle Kriterien erfüllt.
- "Fertig" heißt: Tests, Lint, Typen und Import-Verträge grün, Ausgabe gezeigt.

## Umfang
Vermeide Overengineering. Ändere nur, was verlangt oder zwingend nötig ist.
Keine zusätzlichen Features, kein Refactoring nebenbei, keine neuen Abstraktionen
oder Konfigurationsoptionen für einen einzigen Fall. Validierung nur an
Systemgrenzen. Keine neuen Pakete ohne Zustimmung. Ideen außerhalb des Auftrags
am Ende in einem Satz nennen, nicht umsetzen.

## Belege
Keine Aussagen über ungeöffneten Code. APIs gegen die installierte Version prüfen.
"Läuft" nur mit Befehlsausgabe. Unsicherheit offen sagen.

## Tests
Allgemeine Lösungen, keine hart codierten Sonderfälle. Bestehende Tests nur
ändern, wenn der Auftrag es verlangt; falsche Tests melden statt anpassen.

## Unklarheiten und Architektur
Bei Unklarheiten mit großer Wirkung fragen, kleine mit der naheliegendsten
Annahme lösen und nennen. Abweichungen von ADRs, neue Muster oder Schichten:
anhalten und einen ADR-Entwurf vorlegen.
```

CLAUDE.md

```
@AGENTS.md

## Nur für Claude Code
```

- Für Suchen im Code und Testläufe den Teilagenten "testlaeufer" verwenden.
- Reviews über den Teilagenten "pruefer" (nur lesend).

A.2 Aufgabenstellungen

Feature

Setze FR-012 (docs/anforderungen/FR-012.md) um. Beachte die dort genannten ADRs.
 Erlaubte Pfade: app/notizen/, app/db/, tests/. Keine neuen Abhängigkeiten.
 Erst Plan (IDs, Dateien, Annahmen), dann Tests pro Akzeptanzkriterium, dann Umsetzung.
 Fertig: alle Tests, ruff, mypy und lint-imports grün, Ausgaben gezeigt.
 Nicht Teil der Aufgabe: Suche nach Tags (FR-013).

Bugfix

Fehler: PUT /notizen/{id}/tags liefert 500 bei leerer Liste (Log unten).
 Schreibe zuerst einen Test, der den Fehler reproduziert, und zeige, dass er rot ist.
 Behebe dann die Ursache mit der kleinsten Änderung. Kein Aufräumen daneben.
 Fertig: neuer Test und alle bestehenden Tests grün, Ausgabe gezeigt.

Review durch das zweite Modell

Prüfe git diff main...HEAD gegen docs/anforderungen/FR-012.md und die dort genannten ADRs. Nur belegte Befunde (Datei, Zeile, ID, Fehlerfall):
 fehlende Tests zu Akzeptanzkriterien, Änderungen außerhalb des Auftrags,
 ADR-Verstöße, Fehler. Keine Stilfragen. Ändere keine Dateien.

Goal

/goal Jedes Akzeptanzkriterium von FR-012 hat einen Test mit req("FR-012", ...);
 uv run pytest, uv run ruff check . und uv run lint-imports enden ohne Fehler und
 ihre Ausgabe ist im Verlauf sichtbar; keine bestehende Testdatei wurde geändert;
 keine Änderung an pyproject.toml; höchstens 25 Turns.

Architekturfrage

Wir brauchen Volltextsuche über Notizen (FR-013). Lies ADR-0003 und NFR-004.
 Vergleiche höchstens drei Optionen (etwa PostgreSQL-Volltext, pg_trgm, externer
 Suchdienst) nach Aufwand, Betrieb und Erfüllung von NFR-004. Keine Umsetzung.
 Ergebnis: ADR-Entwurf mit Kontext, Optionen, Empfehlung und "Prüfbar durch".

A.3 Schnellreferenz

	Claude Code	Codex
Modell wählen	<code>/model</code> , <code>--model opus</code>	<code>/model</code> , <code>-m gpt-6.1-sol</code>
Effort	<code>/effort high</code> , <code>--effort low</code>	<code>-c model_reasoning_effort="high"</code>
Einmal gründlicher	ultrathink im Prompt	Max über <i>More reasoning...</i>
Planen	Plan-Modus (<code>⌘</code> <code>Tab</code>)	<code>/plan</code>
Ziel verfolgen	<code>/goal <Bedingung></code>	<code>/goal <Ziel></code> , <code>pause</code> , <code>resume</code>
Review	Teilagent <code>pruefer</code>	<code>/review</code> , <code>exec -s read-only</code>
Kontext leeren	<code>/clear</code> , <code>/compact</code>	neue Unterhaltung, <code>/compact</code>
Parallel arbeiten	<code>claude --worktree <name></code>	Worktree-Modus (App), <code>--worktree</code> (CLI, experimentell)
Sandbox	<code>/sandbox</code>	<code>sandbox_mode</code> , <code>network_access = false</code>
Mutation Testing	<code>uv run mutmut run "<modul>*"</code>	dasselbe
Regeln	CLAUDE.md (importiert AGENTS.md)	AGENTS.md
Konfiguration	<code>.claude/settings.json</code>	<code>.codex/config.toml</code>

B Glossar

Adaptive Reasoning

Das Modell entscheidet pro Schritt, ob und wie lange es nachdenkt; der Effort bestimmt, wie großzügig.

ADR

Architecture Decision Record: kurzes Dokument zu einer Architekturentscheidung mit Kontext und Konsequenzen.

Chat-Vorlage

Format mit Sondertoken, in dem ein Modell System-Prompt, Benutzer- und Antwortteile erwartet.

Akzeptanzkriterium

Prüfbare Bedingung, unter der eine Anforderung als erfüllt gilt; idealerweise ein Test.

AGENTS.md

Regeldatei für Codex (und andere Agenten) im Repository.

CLAUDE.md

Regeldatei für Claude Code; kann andere Dateien per `@pfa` einbinden.

Effort

Regler für die Gründlichkeit des Nachdenkens (Claude: `low` bis `max`, Codex: `Light` bis `Max`).

GGUF

Einzeldatei-Format von llama.cpp mit quantisierten Gewichten, Tokenizer, Metadaten und Chat-Vorlage.

Goal

Zielbedingung, bis zu deren Erfüllung der Agent selbstständig weiterarbeitet (`/goal`).

Halluzination

Plausible, aber unbelegte Behauptung, etwa eine erfundene API oder ein nicht ausgeführter Test.

Hook

Befehl, den das Werkzeug bei einem Ereignis automatisch ausführt, etwa nach jeder Dateiänderung.

import-linter

Python-Werkzeug, das Regeln über erlaubte Importe als Verträge prüft.

KV-Cache

Zwischenspeicher der Aufmerksamkeitswerte bisheriger Token; wächst linear mit der Kontextlänge.

Kontextfenster

Menge an Text, die ein Modell gleichzeitig berücksichtigt; wird bei Überlauf verdichtet.

MLX

Apples Framework für maschinelles Lernen auf Apple Silicon; MLX-Modelle sind Verzeichnisse mit `safetensors`.

MoE

Mixture of Experts: Modell, das pro Token nur einen Teil seiner Parameter aktiviert.

Mutation Testing

Prüft Tests, indem gezielt Fehler in den Code eingebaut werden; überlebende Mutanten zeigen Testlücken.

NFR

Nicht-funktionale Anforderung, etwa zu Antwortzeit, Sicherheit oder Verfügbarkeit.

opusplan

Claude-Code-Alias: Opus im Plan-Modus, Sonnet für die Ausführung.

Overengineering

Mehr Abstraktion, Konfigurierbarkeit oder Funktion als verlangt.

Plan-Modus

Modus, in dem der Agent liest und plant, aber nichts ändert.

Prompt Injection

Anweisungen in gelesenen Inhalten (Dateien, Webseiten, Werkzeugausgaben), die den Agenten manipulieren.

Sandbox

Isolation von Dateisystem und Netzwerk, in der Befehle des Agenten laufen.

Quantisierung

Speichern der Gewichte mit weniger Bits (etwa 4 statt 16), spart Speicher, kostet etwas Qualität.

safetensors

Standardformat für Modellgewichte auf Hugging Face, ohne ausführbaren Code ladbar.

Sampling

Auswahl des nächsten Tokens aus der Wahrscheinlichkeitsverteilung, gesteuert über Temperature, top-p u. a.

Scope Creep

Schleichende Ausweitung einer Änderung über den Auftrag hinaus.

Slopsquatting

Registrierung von Paketnamen, die Modelle erfinden, mit Schadcode.

Spec-Driven Development

Vorgehen, bei dem Spezifikation, Plan und Aufgaben vor dem Code entstehen.

Subagent / Teilagent

Vom Hauptagenten beauftragter Agent mit eigenem Kontext, oft auf kleinerem Modell.

System-Prompt

Anweisungen, die Rolle und Regeln für eine ganze Sitzung festlegen.

Temperature

Parameter, der die Wahrscheinlichkeitsverteilung schärft oder glättet; bei aktuellen Frontier-Modellen gesperrt.

Test-Gaming

Tests werden abgeschwächt oder Sonderfälle hart codiert, damit Prüfungen grün werden.

ultracode / Ultra

Modus, in dem eine große Aufgabe auf parallele Teilagenten verteilt wird.

Worktree

Zusätzliches Arbeitsverzeichnis desselben Repositories mit eigenem Branch, für parallele Agenten.

Quellen und weiterführende Literatur stehen im anschließenden Literaturverzeichnis. Die Begleitbände sind **Git von Grund auf** (CI mit Gitea Actions) [20] und **Secure API Design** (Beispielprojekt, Fehlerformat) [5].

Literaturverzeichnis

- [1] Anthropic, „Models overview“, Claude Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://platform.claude.com/docs/en/models/overview>
- [2] Anthropic, „Model configuration“, Claude Code Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.claude.com/docs/en/model-config>
- [3] OpenAI, „Codex models“, Codex Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://learn.chatgpt.com/docs/models>
- [4] P. Haindl, *Docker von Grund auf*. Persönliches Handbuch, unveröffentlicht, 2026.
- [5] P. Haindl, *Secure API Design mit Python*. Persönliches Handbuch, unveröffentlicht, 2026.
- [6] E. Schluntz und B. Zhang, „Building effective agents“, Anthropic Engineering. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.anthropic.com/engineering/building-effective-agents>
- [7] S. Yao u. a., „ReAct: Synergizing reasoning and acting in language models“, in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, 2023. [Online]. Verfügbar unter: <https://arxiv.org/abs/2210.03629>
- [8] Anthropic, „Prompting best practices“, Claude Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>
- [9] OpenAI, „Developer commands“, Codex Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://learn.chatgpt.com/docs/developer-commands?surface=cli>
- [10] Anthropic, „Prompting Claude Fable 5.1“, Claude Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-claude-fable-5-1>
- [11] Anthropic, „Prompting Claude Opus 5.5“, Claude Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-claude-opus-5-5>
- [12] Anthropic, „Keep Claude working toward a goal“, Claude Code Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.claude.com/docs/en/goal>
- [13] OpenAI, „Codex release 0.154.0“, GitHub: openai/codex. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/openai/codex/releases/tag/rust-v0.154.0>
- [14] Anthropic, „Effort“, Claude Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://platform.claude.com/docs/en/build-with-claude/effort>
- [15] Anthropic, „Migrating to Claude Opus 5.5“, Claude Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://platform.claude.com/docs/en/models/opus-5-5/migration-guide>
- [16] OpenAI, „Config basics“, Codex Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://learn.chatgpt.com/docs/config-file/config-basic>
- [17] Anthropic, „Prompt caching“, Claude Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
- [18] Anthropic, „Subagents“, Claude Code Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.claude.com/docs/en/sub-agents>
- [19] OpenAI, „Configuration reference“, Codex Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://learn.chatgpt.com/docs/config-file/config-reference>
- [20] P. Haindl, *Git von Grund auf*. Persönliches Handbuch, unveröffentlicht, 2026.
- [21] OpenAI, „Agent approvals & security“, Codex Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://learn.chatgpt.com/docs/agent-approvals-security>
- [22] Git development community, „git-worktree: Manage multiple working trees“, Git Documentation. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://git-scm.com/docs/git-worktree>
- [23] Anthropic, „Run parallel sessions with worktrees“, Claude Code Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.claude.com/docs/en/worktrees>
- [24] Z. Ji u. a., „Survey of hallucination in natural language generation“, *ACM Computing Surveys*, Bd. 55, Nr. 12, S. 1–38, 2023, doi: [10.1145/3571730](https://doi.org/10.1145/3571730).
- [25] OpenAI, „Docs MCP“, OpenAI Developers. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developers.openai.com/learn/docs-mcp>

- [26] Anthropic, „Choose a permission mode“, Claude Code Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.claude.com/docs/en/permission-modes>
- [27] Anthropic, „Week 20 · May 11–15, 2026“, Claude Code Docs: What's new. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.claude.com/docs/en/whats-new/2026-w20>
- [28] OpenAI, „Using Goals in Codex“, OpenAI Cookbook. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://developers.openai.com/cookbook/examples/codex/using_goals_in_codex
- [29] Anthropic, „Hooks reference“, Claude Code Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.claude.com/docs/en/hooks>
- [30] GitHub, „Spec Kit: Toolkit for spec-driven development“, GitHub repository. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/github/spec-kit>
- [31] Kiro, „Specs“, Kiro Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://kiro.dev/docs/specs/>
- [32] Locust contributors, „Locust documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.locust.io/en/stable/>
- [33] M. Nygard, „Documenting architecture decisions“, Cognitect Blog. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- [34] ADR GitHub Organization, „MADR: Markdown Architectural Decision Records“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://adr.github.io/madr/>
- [35] M. Nottingham, E. Wilde, und S. Dalal, „Problem Details for HTTP APIs“, RFC9457, Juli 2023. [Online]. Verfügbar unter: <https://www.rfc-editor.org/info/rfc9457>
- [36] pytest development team, „How to mark test functions with attributes“, pytest Documentation. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/how-to/mark.html>
- [37] Import Linter contributors, „Import Linter“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://import-linter.readthedocs.io/>
- [38] S. Brown, „The C4 model for visualising software architecture“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://c4model.com/>
- [39] Astral, „uv“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.astral.sh/uv/>
- [40] Astral, „Ruff“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.astral.sh/ruff/>
- [41] Mypy contributors, „Mypy documentation“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://mypy.readthedocs.io/en/stable/>
- [42] Python Packaging Authority, „pip-audit: Audit Python environments and dependency trees for known vulnerabilities“, GitHub repository. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/pypa/pip-audit>
- [43] Y. Jia und M. Harman, „An analysis and survey of the development of mutation testing“, *IEEE Transactions on Software Engineering*, Bd. 37, Nr. 5, S. 649–678, Sep. 2011, doi: [10.1109/TSE.2010.62](https://doi.org/10.1109/TSE.2010.62).
- [44] boxed, „mutmut: Python mutation testing system“, GitHub repository. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/boxed/mutmut>
- [45] C. Foster u. a., „Mutation-guided LLM-based test generation at Meta“, in *Proc. ACM Int. Conf. Foundations of Software Engineering (FSE), Industry Track*, 2025. [Online]. Verfügbar unter: <https://arxiv.org/abs/2501.12862>
- [46] OpenAI, „Hooks“, Codex Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://learn.chatgpt.com/docs/hooks>
- [47] Anthropic, „Security“, Claude Code Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.claude.com/docs/en/security>
- [48] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, und M. Fritz, „Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection“, 2023. [Online]. Verfügbar unter: <https://arxiv.org/abs/2302.12173>
- [49] S. Willison, „The lethal trifecta for AI agents: private data, untrusted content, and external communication“, Simon Willison's Weblog. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
- [50] Anthropic, „Configure the sandboxed Bash tool“, Claude Code Docs. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.claude.com/docs/en/sandboxing>
- [51] MITRE, „CVE-2025-59536: Claude Code's startup trust dialog could lead to command execution“, CVE Program. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.cve.org/CVERecord?id=CVE-2025-59536>
- [52] T. Claburn, „LLMs can't stop making up software dependencies and sabotaging everything“, The Register. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://www.theregister.com/2025/04/12/ai_code_suggestions_sabotage_supply_chain/

- [53] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, und M. Jadliwala, „We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs“, in *Proc. 34th USENIX Security Symposium*, 2025. [Online]. Verfügbar unter: <https://arxiv.org/abs/2406.10279>
- [54] Anthropic, „How Claude remembers your project“, Claude Code Docs. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://code.claude.com/docs/en/memory>
- [55] OpenAI, „AGENTS.md“, Codex Docs. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://learn.chatgpt.com/docs/agent-configuration/agents-md>
- [56] A. Holtzman, J. Buys, L. Du, M. Forbes, und Y. Choi, „The curious case of neural text degeneration“, in *Proc. 8th Int. Conf. Learning Representations (ICLR)*, 2020. [Online]. Verfügbar unter: <https://arxiv.org/abs/1904.09751>
- [57] M. N. Nguyen, A. Baker, C. Neo, A. Roush, A. Kirsch, und R. Schwartz-Ziv, „Turning up the heat: Min-p sampling for creative and coherent LLM outputs“, in *Proc. 13th Int. Conf. Learning Representations (ICLR)*, 2025. [Online]. Verfügbar unter: <https://arxiv.org/abs/2407.01082>
- [58] OpenAI, „Latest model guide“, OpenAI API Docs. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developers.openai.com/api/docs/guides/latest-model>
- [59] OpenAI, „Reasoning models“, OpenAI API Docs. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://developers.openai.com/api/docs/guides/reasoning>
- [60] N. F. Liu u. a., „Lost in the middle: How language models use long contexts“, *Transactions of the Association for Computational Linguistics*, Bd. 12, S. 157–173, 2024, [Online]. Verfügbar unter: <https://arxiv.org/abs/2307.03172>
- [61] Hugging Face, „Safetensors“, Hugging Face Documentation. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://huggingface.co/docs/safetensors/index>
- [62] ml-explore, „MLX LM: Run LLMs with MLX“, GitHub repository. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/ml-explore/mlx-lm>
- [63] Hugging Face, „GGUF“, Hugging Face Hub Documentation. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://huggingface.co/docs/hub/gguf>
- [64] ggml-org, „llama.cpp: LLM inference in C/C++“, GitHub repository. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/ggml-org/llama.cpp>
- [65] TerminalBytes, „Ollama vs llama.cpp vs MLX on a Mac Studio, measured“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://terminalbytes.com/ollama-vs-llama-cpp-vs-mlx-mac-2026/>
- [66] Ollama, „Ollama is now powered by MLX on Apple Silicon in preview“, Ollama Blog. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://ollama.com/blog/mlx>
- [67] LM Studio, „LM Studio 0.3.4 ships with Apple MLX“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://lmstudio.ai/mlx>
- [68] E. Frantar, S. Ashkboos, T. Hoefler, und D. Alistarh, „GPTQ: Accurate post-training quantization for generative pre-trained transformers“, in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, 2023. [Online]. Verfügbar unter: <https://arxiv.org/abs/2210.17323>
- [69] J. Lin u. a., „AWQ: Activation-aware weight quantization for LLM compression and acceleration“, in *Proc. 7th Conf. Machine Learning and Systems (MLSys)*, 2024. [Online]. Verfügbar unter: <https://arxiv.org/abs/2306.00978>
- [70] W. Kwon u. a., „Efficient memory management for large language model serving with PagedAttention“, in *Proc. 29th ACM Symp. Operating Systems Principles (SOSP)*, 2023. [Online]. Verfügbar unter: <https://arxiv.org/abs/2309.06180>
- [71] N. Shazeer u. a., „Outrageously large neural networks: The sparsely-gated mixture-of-experts layer“, in *Proc. 5th Int. Conf. Learning Representations (ICLR)*, 2017. [Online]. Verfügbar unter: <https://arxiv.org/abs/1701.06538>
- [72] Ollama, „FAQ“, Ollama Docs. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.ollama.com/faq>
- [73] Qwen Team, „Qwen3-32B: Best practices“, Hugging Face Model Card. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://huggingface.co/Qwen/Qwen3-32B>
- [74] Qwen Team, „Qwen3 technical report“, 2025. [Online]. Verfügbar unter: <https://arxiv.org/abs/2505.09388>
- [75] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 4. Aufl. Thousand Oaks, CA, USA: SAGE Publications, 2018.
- [76] LM Studio, „Claude Code“, LM Studio Docs: Integrations. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://lmstudio.ai/docs/integrations/claude-code>